Cont/x: A Low-Level Keyboard-Hook Lockdown and On-Device Compiler-Based Assessment Engine for Secure Programming Exams

---

A Capstone Project Presented to the Faculty of

the School of Computer Studies

University of San Jose - Recoletos

Cebu City, Philippines

In Partial Fulfillment

of the Requirements for the Degree of

Bachelor of Science in Information Technology

By

Brian Joseph B. Aratia

Robert Emmanuel C. Avelino

Mel Stephen A. Perez

Adviser

Engr. Violdan Bayocot

Table of Contents

CHAPTER 1 ...1

Introduction

Rationale ... 1

Review of Related Literature ... 2

Review of Related Works ... 4

Project Objectives: ... 7

General Goals ... 7

Specific Goals ... 8

Significance of the Study ... 9

Project Scope and Limitations ... 10

CHAPTER 2 ...11

Software Requirements and Design Specifications

Application Overview ... 11

Use Case Diagram ... 12

Architectural Diagram ... 13

Use Case Narratives (Student) ... 15

Use Case Narratives (Teacher) ... 26

Use Case Narratives (Admin) ... 39

Activity Diagram (Student) ... 45

Activity Diagram (Teacher) ... 51

Activity Diagram (Admin) ... 57

Entity Relationship Diagram ... 63

Class Diagram ... 64

User Interface Design (Student) ... 65

User Interface Design (Teacher) ... 71

User Interface Design (Admin) ... 79

REFERENCES ...82

CHAPTER 1

Introduction

Rationale

Programming examinations in computing education increasingly rely on computer-based delivery, making the integrity and reliability of exam systems a critical concern for computing instructors. This study addresses a dual challenge: the need to secure the entire Windows desktop environment during high-stakes programming exams and the need to deliver fast, reliable, and consistent automated grading of code, particularly in settings where network infrastructure may be unstable. The key areas of concentration include system-wide lockdown mechanisms, on-device automated code assessment, and cloud-based exam management, integrated into a single cohesive application. This topic is worth investigating because programming examinations occupy a unique position among academic assessments: they are simultaneously high-stakes, technically demanding to grade, and highly susceptible to academic dishonesty in ways that conventional proctoring methods were never designed to address [1].

This results in two central problems. First, most lockdown tools do not fully secure the Windows desktop. They may block a browser tab or a few shortcuts, but without low-level keyboard interception and process monitoring, a student can still switch to an unauthorized application undetected. Second, because grading is done on a remote server, code has to travel over the internet to be compiled and checked. This causes the exam to slow down badly when many students submit at the same time, and it can fail completely if the network is weak or unstable. This matters because when the system lags or drops connection, students lose time they cannot get back, and teachers lose confidence that the results reflect real ability rather than

technical failure. A tool that closes both gaps, namely weak desktop lockdown and slow server-based grading, is needed to make programming exams fair, fast, and reliable for both students and teachers.

Contx is proposed as a Windows desktop and web-based platform that integrates system-wide exam lockdown, local code compilation and automated assessment, multi-type exam delivery, and real-time teacher monitoring into a single solution tailored for computing programs. Teachers build and manage exams through a web portal, while students use a locked-down Windows client to answer both traditional item types and live coding questions, with all exam data synchronized through the cloud in real time.

Review of Related Literature

Academic Dishonesty in Programming Education

Academic dishonesty is not a marginal issue in computing education but a documented and pervasive one. A review of cheating patterns in university-level computer science courses reports that in numerous programming classes, more than half of enrolled students admitted to engaging in some form of academic dishonesty [1]. Programming assessments are particularly exposed to this risk because source code is easy to copy, modify, and disguise, and because the correctness of a submission can often be verified without the grader ever needing to understand how the student arrived at the solution. This creates an incentive structure in which shortcuts are both technically easy to take and difficult for an instructor to detect through manual inspection alone, underscoring the need for testing environments that constrain what a student can access while an exam is in progress.

The Rise of AI-Assisted Cheating

The academic dishonesty problem has been compounded by the rapid adoption of generative artificial intelligence tools capable of producing working code on demand. Survey-based research conducted shortly after the public release of ChatGPT found that a substantial share of secondary and tertiary students were already using such tools for academic tasks, and that familiarity with and reliance on these tools continued to grow across subsequent school years [2]. For programming courses specifically, this trend is significant because generative AI tools can produce syntactically correct, functioning solutions to typical exam-style problems within seconds, effectively neutralizing the difficulty that once made programming exams self-policing. This shifts the burden of academic integrity away from the difficulty of the task itself and onto the testing environment, reinforcing the need for lockdown mechanisms capable of restricting access to such tools during an active examination.

Connectivity Constraints in Online Assessment

A parallel and often overlooked concern in the literature is the effect of infrastructure limitations on the delivery of computer-based assessments. Studies of institutions operating under constrained connectivity have documented that unstable bandwidth measurably interferes with students' ability to complete time-bound online assessments, independent of how well-designed the assessment platform itself may be [3]. This finding is particularly relevant to computing programs in the Philippines and other developing regions, where campus and residential internet service can be inconsistent. Literature in this area suggests that assessment integrity and assessment reliability cannot be treated as purely a security problem, since a student

who loses connectivity mid-exam faces the same practical harm as one who was never fairly evaluated to begin with.

Scalability Challenges in Automated Code Grading

The literature on automated code assessment further identifies scalability as a structural weakness of server-dependent grading. A review of automated code assessment techniques and tools notes that even dedicated code execution services can be overwhelmed during concentrated exam-period demand, requiring throttling or queuing mechanisms that delay feedback to students [4]. Because programming exams typically require dozens or hundreds of students to submit and compile code within the same narrow window, centralized execution backends are especially prone to this kind of contention. This body of literature motivates the case for on-device compilation, which removes the dependency on a shared execution backend entirely and allows assessment to scale linearly with the number of student machines rather than the capacity of a central server

Review of Related Works

One widely used tool for securing computer-based exams is Respondus LockDown Browser, a testing environment developed by Respondus, Inc. that prevents students from copying, printing, accessing other applications, or visiting outside websites for the duration of an exam [5]. A comparative review of lockdown tools found that products like LockDown Browser consistently block known methods of digital cheating, including virtual machines, remote desktop access, and screen recording, though the same review also noted device-compatibility issues and the need for administrative setup before each exam [13]. LockDown Browser is

effective at restricting the testing device itself, but like other tools in its category, it is not built to evaluate whether the code a student writes is actually correct, leaving grading completely outside its scope.

A similar tool is Safe Exam Browser (SEB), which turns a regular computer into a secure workstation by blocking access to system functions, other applications, and websites while an assessment is in progress [6]. SEB is open-source and widely adopted by universities because it can be configured to work alongside existing learning management systems without requiring a separate paid license. Like LockDown Browser, however, SEB's role ends at restricting the device, it does not compile, run, or check the correctness of any code a student submits, so institutions using it for programming exams still need a separate grading system on top of it.

Beyond simple browser lockdown, some platforms add behavioral monitoring to watch the test-taker rather than just the device. Proctorio is one such tool, using machine learning and facial recognition to track a test-taker's webcam, microphone, and screen activity throughout an exam, flagging behavior such as looking away from the screen or unusual audio for later review [7], [8], [9]. Reporting on Proctorio has raised concerns that institutions must weigh its cost, its compatibility with assistive technology used by students with disabilities, and the privacy implications of continuous audio-visual recording during an exam [10]. These concerns are relevant to Contx’s decision to rely on software-level restrictions, such as keyboard interception and process monitoring, rather than webcam or biometric proctoring.

On the grading side, CodeChum is an online learning and monitoring platform that lets students submit C programming exercises through a web-based environment, with each submission checked against predefined test cases and the results returned automatically [11]. It is

documented to reduce instructor workload and give students faster feedback than manual grading allows, but it relies on centralized, server-side execution of student code, which, based on the scalability concerns already discussed, leaves it open to slowdowns or interruptions when connectivity is unstable or when many students submit at the same time.

CodeRunner, a plugin for the Moodle learning management system, works in a similar way. It lets instructors write questions that require a code submission, which the system then runs against test cases to determine correctness and assign marks without manual grading [12]. Because it is built as a Moodle plugin rather than a standalone application, CodeRunner depends entirely on Moodle's own server infrastructure to execute and grade code, which means the same network and server-load problems that affect CodeChum apply here as well, and neither tool includes any mechanism for locking down the student's desktop during an exam.

Taken together, these studies reveal a consistent divide in how existing tools approach programming exam security. The existing lockdown systems all focus on restricting or monitoring the exam-taking environment, but none of them touch the correctness of the code a student produces, grading is treated as someone else's problem. CodeChum and CodeRunner sit on the opposite side of that divide, though both automate the evaluation of code effectively, neither includes any mechanism for locking down the student's desktop, and both depend entirely on centralized, server-side execution that becomes unreliable under heavy load or unstable connectivity. This gap is precisely what Contx is designed to close, bringing keyboard interception and process monitoring together with local compilation and grading, so that exam security and exam evaluation are no longer handled by two disconnected systems.

Project Objectives

The project aims to develop an integrated desktop and web-based solution that addresses academic dishonesty and infrastructure instability in programming examinations by combining system-wide lockdown enforcement with on-device, compiler-based code assessment. By restricting unauthorized access to the device while simultaneously compiling and verifying student submissions locally, the platform seeks to eliminate the dependency on continuous network connectivity and centralized execution servers that limit existing lockdown browsers and automated grading systems. Through the integration of a cloud-based management portal for instructors and administrators, Cont/x further aims to centralize exam authentication, distribution, and record-keeping, thereby offering a locally deployable model for secure programming assessment.

General Goals:

1. **Secure the Examination Environment:** Deliver a system-wide lockdown that shields the entire Windows Desktop from unauthorized applications and shortcuts throughout an active exam.
2. **Enable Reliable On-Device Assessment:** Provide instant, accurate grading of C programming submissions thorough local compilation, independent of internet connectivity.
3. **Centralize Exam Oversight:** Give instructors a cloud-based portal to build, distribute, and monitor exams with confidence in their integrity.

4. **Unify Security and Evaluation:** Bring lockdown enforcement and automated grading together into one cohesive platform, closing the gap left by tools that handle only one side of the problem

Specific Goals:

Functional Goals

- **System-Wide Lockdown Enforcement**: Design and implement a lockdown mechanism that restricts unauthorized applications, system shortcuts, and external resources on the student's device throughout the duration of an active examination.
- **On-Device Automated Assessment Engine**: Develop an assessment engine capable of compiling, executing, and verifying student C programming submissions directly on the local machine, without requiring continuous server-side processing.
- **Cloud-Based Exam Management Portal**: Build a web platform that allows instructors to create, distribute, and manage programming examinations, while centralizing authentication and assessment records for administrators.
- **Component Integration and Synchronization**: Integrate the lockdown, assessment, and cloud-management components into a single cohesive application that automatically synchronizes exam data once internet connectivity becomes available.

Non-Functional Goals

- **Reliability & Availability**: Ensure that lockdown enforcement and on-device assessment operate consistently and without interruption, regardless of the stability of the student's internet connection.
- **Performance & Responsiveness**: Minimize response latency during local compilation and code verification so that students receive assessment feedback immediately upon submission.
- **Security**: Enforce software-level restrictions, such as keyboard interception and process monitoring, to prevent students from bypassing the lockdown environment during examinations.
- **Usability & Accessibility**: Provide an intuitive interface for both students and instructors that requires minimal technical proficiency to operate the lockdown client and the web-based exam portal.

Significance of the Study

This study will be beneficial to the following:

Students. The students taking up computer related programs will benefit from a secure and standardized testing environment during programming examinations, reducing opportunities for academic dishonesty while ensuring that their performance is evaluated fairly and consistently. The automated assessment of C programming submissions will also allow students to receive immediate feedback on their code, rather than waiting for manual checking.

Instructors. Instructors will benefit from the reduced manual workload associated with checking and grading programming examinations, as the system will automatically compile and assess student code. The exam lockdown feature will also give instructors greater confidence in the integrity of student submissions, while the monitoring function will allow them to track student progress during an ongoing exam without needing to manually patrol the examination room.

Project Scope and Limitations

The purpose of this study is to design, develop, and evaluate Cont/x, a Windows desktop and web application intended for secure exam lockdown and automated assessment of programming examinations written in the C programming language, for use in computing education.

- This study will only focus on the design, development, and evaluation of the Cont/x Windows desktop client and its accompanying web application. It will not address platforms other than Windows, such as macOS, Linux, or mobile operating systems.
- Only the C programming language will be supported for local compilation and automated grading, and not any other programming language.
- The lockdown features of the system will be limited to software-level restrictions, such as keyboard interception and process monitoring, and will not include webcam-based, biometric, or AI-driven proctoring.
- The system will not include plagiarism or code-similarity detection, nor will it integrate with existing learning management systems.

CHAPTER 2

Software Requirements and Design Specifications

Application Overview

Contx is an integrated Windows desktop and web-based platform designed to secure programming examinations while enabling fast, reliable, on-device assessment of student code. It is composed of two connected interfaces, a locked-down desktop client for students during an active exam, and a web-based portal for teachers and administrators, both synchronized through a centralized cloud backend.

The desktop client runs as an Electron application with a React interface and an embedded Monaco code editor. Lockdown takes effect immediately on launch, an OS lockdown module intercepts keyboard input, mutes system audio, and monitors running processes to detect and automatically terminate unauthorized applications for the remainder of the exam. Submissions are compiled and executed locally through GCC via MinGW, giving students immediate feedback without a remote grading server. Exam data and violation logs are cached locally through better-sqlite3, allowing the client to keep working through connectivity loss and sync automatically once a connection returns.

The web portal, built on React with a Node.js and Express API, serves two roles. Teachers create and distribute exams, monitor student progress in real time, and review violation logs. Administrators are limited to CRUD operations on teacher and student accounts, keeping account management separate from exam content and grading.

Both interfaces connect to a shared Supabase backend, which handles authentication for exam access codes, stores exams and scores, synchronizes data in real time, and manages cloud storage for exported files. This keeps exam security and exam evaluation unified in one system rather than split across disconnected tools.

Use Case Diagram

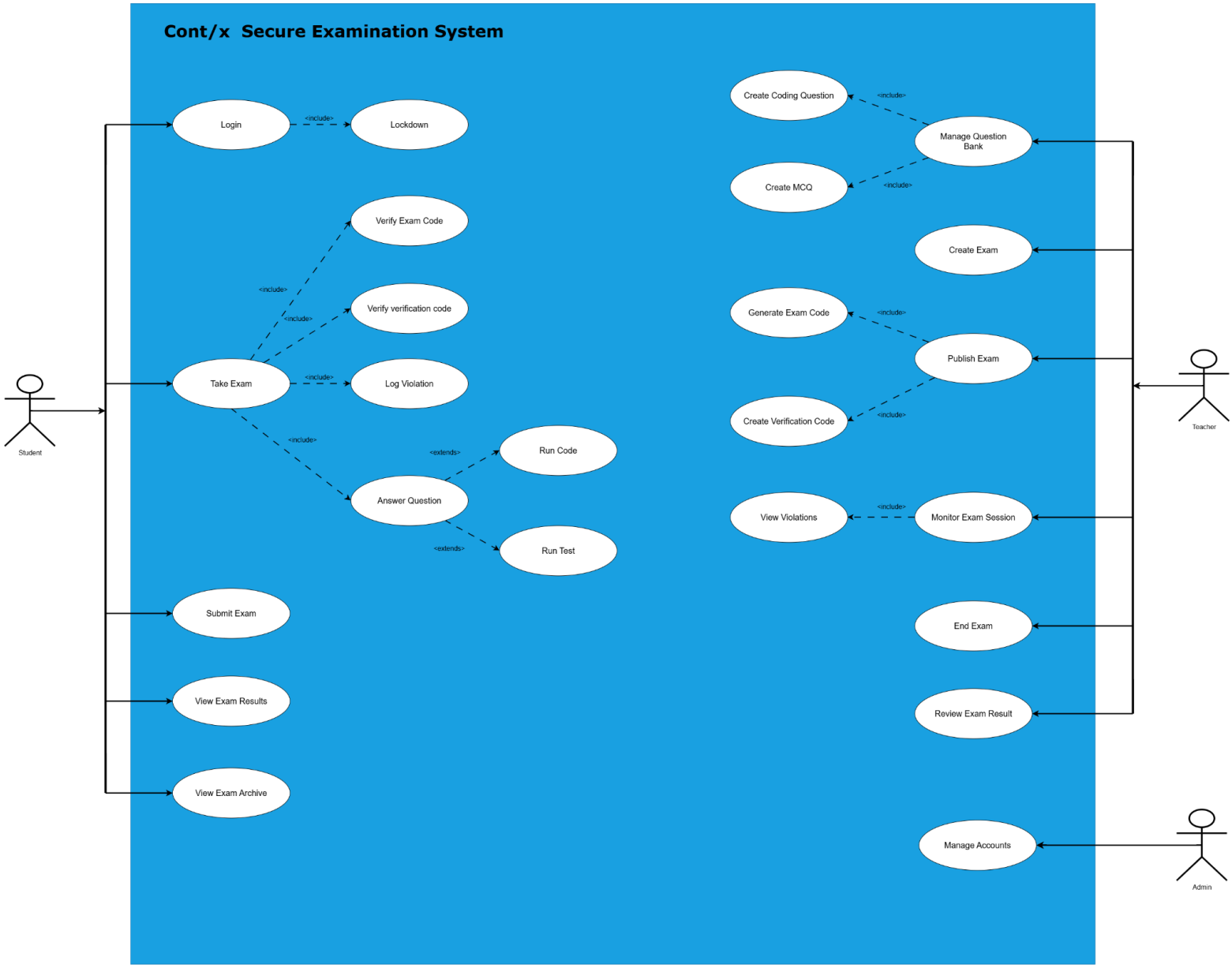


Figure 1: Contx Secure Examination System

Figure 1 illustrates the core use case of the Cont/x Secure Examination System. It highlights how each user role, Student, Teacher, and Admin interacts with the system's features such as exam access and verification, question answering with code execution and testing, exam question authoring and publishing, live session monitoring with violation tracking, and account management. This diagram serves as a foundational overview of the system's functional requirements and user responsibilities.

Architectural Diagram

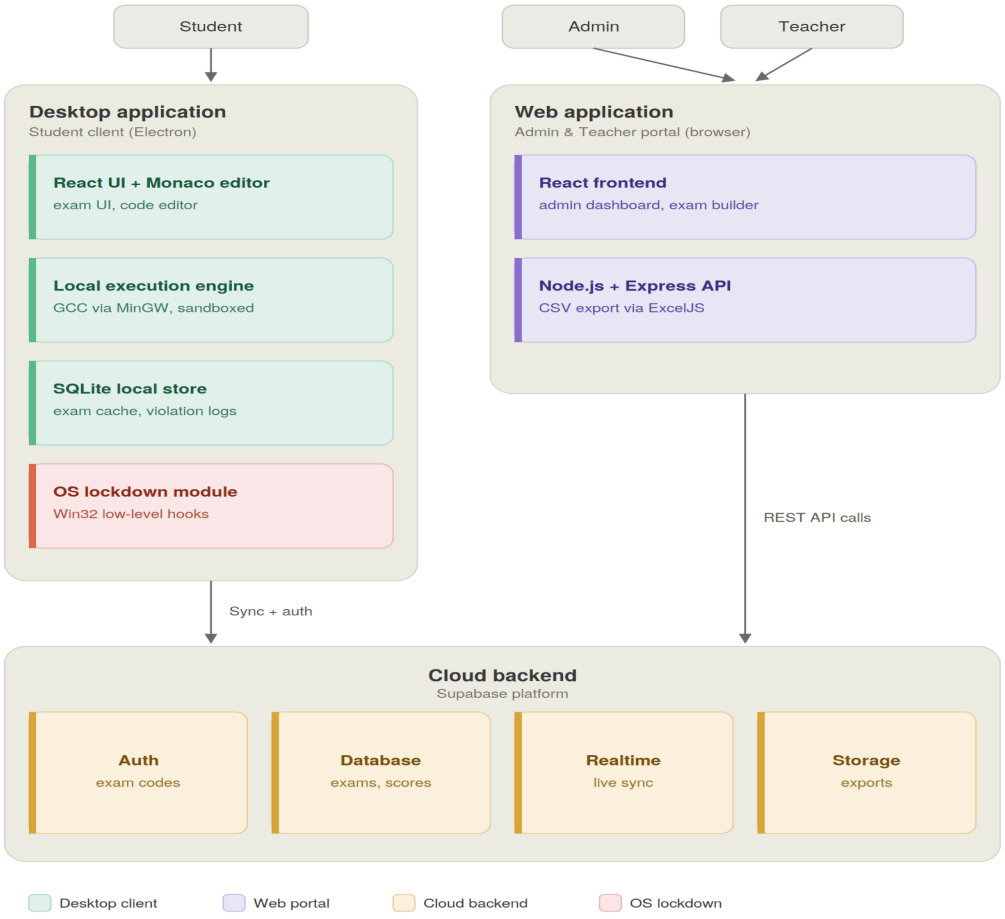


Figure 2 - Architectural Diagram

The architectural design in Figure 2 illustrates the flow of interaction between the three user roles, Student, Admin, and Teacher, across the desktop and web interfaces, connected through a centralized cloud backend. The student interacts with the system through a desktop application built on Electron, which houses the React UI with an embedded Monaco code editor for the exam interface, a local execution engine that compiles and runs submissions locally through GCC via MinGW within a sandboxed environment, an SQLite local store for caching exam data and violation logs, and an OS lockdown module that enforces system-wide restrictions through low-level Win32 hooks. This local architecture allows the desktop client to operate independently of network conditions during an active examination.

The admin and teacher, in contrast, access the platform through a browser-based web application. Its React frontend serves as the shared portal for administrative oversight and exam-building functions, while a Node.js and Express API backend handles server-side logic, including CSV export of exam records through ExcelJS. Both the desktop client and the web application ultimately connect to a centralized cloud backend built on the Supabase platform, which provides authentication for exam codes, a relational database for storing exams and scores, a realtime service for live data synchronization, and cloud storage for exported files. The desktop client communicates with this backend through a sync-and-authentication channel that reconciles locally cached exam and violation data once connectivity is available, while the web application communicates through REST API calls for exam creation, monitoring, and reporting.

Use Case Narratives (Student)

Use Case 1: Student Login

Use Case	Student Login
Actors	Student
Pre-condition	The student has been issued an ID number and temporary password by the system administrator. Cont/x does not support self-registration as all teacher and student accounts are created by an admin.
Post-Condition	The student is authenticated and redirected to the Dashboard, with system lockdown already active.
Flow of Events	
Actor action	System Response
1. The student opens the Cont/x desktop application. 2. The student enters their ID number and password. 3. The student submits the login form.	1. The system activates lockdown and displays the login form requesting an ID Number and Password. . 2. The system looks up the account by ID number, authenticates the credentials, and retrieves the student's profile. 3. The system redirects the student to the Dashboard, which displays an input box for entering the exam code and the 3 latest exams taken.

Alternative Scenarios	
Actor action	System Response
1a. The student enters an invalid ID number or password. 2a. A teacher or admin attempts to log in through the desktop application.	1a. The system displays a generic "Invalid ID number or password" message without specifying which field is incorrect. 2a. The system blocks the login and displays a message instructing them to use the web portal instead.

Table 1. Student Login

Use Case 2: Take Exam

Use Case	Take Exam
Actors	Student
Pre-condition	The student is logged in and on the Dashboard. The teacher has published the exam and generated an exam code and verification code, both of which have been given to the student.
Post-Condition	The student is actively working through the exam, with answers saved as they go, until the exam is submitted or the time limit expires. .
Flow of Events	
Actor action	System Response
1. The student enters the exam code into the input box on the Dashboard.	1. The system verifies the exam code if the exam is active or published.

2. The student submits the exam code. 3. The student enters the verification code into the modal. 4. The student submits the verification code. 5. The student navigates between questions and answers each one.	2. The system checks whether an in-progress exam session already exists for this student on this exam. 3. Finding none, the system displays the exam verification modal, prompting the student for the verification code. 4. The system verifies the code against the exam's verification code 5. The system creates the exam session, closes the modal, starts the exam timer, and loads the exam interface. 6. The system saves each answer as it is submitted.
Alternative Scenarios	
Actor action	System Response
1a. The student enters an invalid or non-existent exam code. 2a. The student enters a code for an exam that has already ended. 3a. The student has already used their maximum allowed attempts for this exam. 4a. An in-progress session already exists for this student on this exam (e.g. the exam was interrupted by an emergency or technical issue). 5a. The student enters an incorrect verification code. 6a. At any point while the exam session is active, the lockdown detects a bypass attempt	1a. The system displays an error message and keeps the student on the Dashboard 2a. The system displays a message informing the student the exam session has closed. 3a. The system displays a message informing the student no further attempts are permitted. 4a. The system skips the verification modal, restores the student's previously saved answers, and redirects the student directly into the exam where they left off. 5a. The system displays an "Invalid verification code" message and keeps the modal open for another attempt. 6a. The system silently logs the violation with

	a timestamp against the session and flags it on the teacher's Violation Monitoring Screen, without interrupting the student.

Table 2. Take Exam

Use Case 3: Answer Question

Use Case	Answer Question
Actors	Student
Pre-condition	The student is inside an active exam session, with the exam interface loaded and the timer running.
Post-Condition	The student's response to the current question is saved, and the student may move to another question.
Flow of Events	
Actor action	System Response
1. The student selects a question from the question navigator. 2. The student provides an answer: selects an option (MCQ), types a response (fill-in-the-blank), or writes code in the editor (coding). 3. For coding questions, the student may run their code against test cases before finalizing.	1. The system displays the question prompt, along with its type-specific input (MCQ choices, fill-in-the-blank field, or code editor). 2. The system displays the run/test results in the editor panel. 3. The system saves the student's current answer against the question and session.

4. The student moves to another question.	
Alternative Scenarios	
Actor action	System Response
1a. The student leaves a question unanswered and navigates away. 2a. The student changes their answer after already answering. 3a. The student's connection drops while answering.	1a. The system saves no answer for that question and marks it as unanswered in the navigator. 2a. The system overwrites the previously saved answer with the latest one. 3a. The system saves the answer locally and queues it for sync once connectivity is restored, without blocking the student from continuing.

Table 3. Answer Question

Use Case 4: Run Code

Use Case	Run Code
Actors	Student
Pre-condition	The student is answering a coding question, with code written in the editor.
Post-Condition	The student's code has been compiled and run against all test cases (visible and hidden), with a check attempt recorded and any configured point deduction applied.
Flow of Events	

Actor action	System Response
1. The student clicks "Run Code" in the editor.	1. The system compiles the student's code locally. 2. The system executes the compiled code against all test cases for the question, both visible and hidden. 3. The system compares each actual output to the expected output and records a pass/fail per test case. 4. The system increments the student's check attempt count for this question and applies the configured point deduction. 5. The system displays the results to the student, showing pass/fail for visible test cases only.
Alternative Scenarios	
Actor action	System Response
1a. The student's code fails to compile. 2a. The student runs code repeatedly, accumulating deductions.	1a. The system displays the compiler error output to the student without running any test cases. 2a. The system continues to apply the configured deduction on each attempt.

Table 4. Run Code

Use Case 5: Run Test

Use Case	Run Test
Actors	Student
Pre-condition	The student is answering a coding question, with code written in the editor.
Post-Condition	The student's code has been compiled and run against the visible test cases only, with results shown for practice purposes.
Flow of Events	
Actor action	System Response
1. The student clicks "Run Test" in the editor.	1. The system compiles the student's code locally. 2. The system executes the compiled code against the visible test cases for the question only. 3. The system compares each actual output to the expected output and records a pass/fail per test case. 4. The system displays the pass/fail results to the student.
Alternative Scenarios	
Actor action	System Response
1a. The student's code fails to compile. 2a. The student runs code repeatedly, accumulating deductions.	1a. The system displays the compiler error output to the student without running any test cases.

	2a. The system continues to apply the configured deduction on each attempt.

Table 5. Run Test

Use Case 6: Submit Exam

Use Case	Submit Exam
Actors	Student
Pre-condition	The student is inside an active exam session, with at least some questions answered. .
Post-Condition	The exam session is marked as submitted, all coding answers are auto-graded, a final score is computed, and the student is redirected to the Exam Result Screen.
Flow of Events	
Actor action	System Response
1. The student clicks "Submit Exam." 2. The student confirms the submission. 3. The student clicks the button to return to the Dashboard.	1. The system displays a confirmation prompt, warning that submission is final. 2. The system marks the exam session as submitted and records the submission timestamp. 3. The system compiles and runs each

	coding answer against all test cases, visible and hidden. 4. The system scores MCQ and fill-in-the-blank answers by comparing them against the teacher-configured correct answers. 5. The system applies any accumulated point deductions from check attempts to the relevant coding questions. 6. The system computes the total score for the session 7. The system redirects the student to the Exam Result Screen, displaying the time taken, the student's score out of the exam's total score, and the number of questions answered correctly out of the total. 8. The system redirects the student to the Dashboard.
Alternative Scenarios	
Actor action	System Response
1a. The student attempts to submit with unanswered questions remaining. 2a. The exam's time limit expires before the student manually submits. 3a. The student loses connection at the moment of submission.	1a. The system warns the student that some questions are unanswered before asking for final confirmation. 2a. The system automatically triggers submission on the student's behalf, marks the session as timed out, and shows the same Exam Result Screen. 3a. The system queues the submission locally

	and completes it automatically once connectivity is restored, then shows the Exam Result Screen once synced.

Table 6. Submit Exam

Use Case 7: View Exam Result

Use Case	View Exam Result
Actors	Student
Pre-condition	The student is logged in and on the Dashboard, which lists exams the student has taken, each with an action button.
Post-Condition	The Exam Result Screen is displayed for the selected exam, showing time taken, score, and correct answer count.
Flow of Events	
Actor action	System Response
1. The student views the Dashboard, which lists their exams. 2. The student clicks the action button for a completed exam. 3. The student clicks the button to return to the Dashboard.	1. The system displays the 3 latest exams with an action button. 2. The system retrieves the session's results and redirects the student to the Exam Result Screen. 3. The system displays the time taken, the student's score out of the exam's total score, and the number of questions answered correctly out of

	the total. 4. The system redirects the student to the Dashboard.
Alternative Scenarios	
Actor action	System Response

Table 7. View Exam Result

Use Case 8: View Exam Archive

Use Case	View Exam Archive
Actors	Student
Pre-condition	The student is logged in, with at least one previously taken exam on record.
Post-Condition	The Archive Screen is displayed, listing all exams the student has taken.
Flow of Events	
Actor action	System Response
1. The student clicks "Archive" in the sidebar navigation. 2. The student clicks on an exam in the list.	1. The system retrieves all exam sessions associated with the student. 2. The system redirects the student to the Archive Screen and displays a list of all exams taken. 3. The system redirects the student to the

	Exam Result Screen for that exam.
Alternative Scenarios	
Actor action	System Response
1a. The student has not taken any exams yet. 2a. The student has multiple attempts recorded for the same exam.	1a. The system displays a message indicating no exams have been taken, instead of a list. 2a. The system lists each attempt separately, or lists the exam once showing the most recent attempt, depending on configuration.

Table 8. View Exam Archive

Use Case Narratives (Teacher)

Use Case 01: Teacher Login

Use Case	Teacher Login
Actors	Teacher
Pre-condition	The teacher has been issued an ID number and temporary password by the system administrator. Cont/x does not support self-registration; all teacher and student accounts are created by an admin.
Post-Condition	The teacher is authenticated and redirected to the Dashboard.
Flow of Events	
Actor action	System Response

1. The teacher opens the Cont/x web portal and enters their ID number and password. 2. The teacher submits the login form.	1. The system displays the login form requesting an ID Number and Password. 2. The system looks up the account by ID number, authenticates the credentials, and retrieves the teacher's profile. 3. The system redirects the teacher to the Dashboard.
Alternative Scenarios	
Actor action	System Response
1a. The teacher enters an invalid ID number or password. 2a. A student attempts to log in through the web portal.	1a. The system displays a generic "Invalid ID number or password" message without specifying which field is incorrect. 2a. The system blocks the login and displays a message instructing the student to use the desktop application instead.

Table 9. Teacher Login

Use Case 02: Create New Exam

Use Case	Create New Exam
Actors	Teacher
Pre-condition	The teacher is logged in and viewing the Exam Library.
Post-Condition	A new exam is created with "draft" status, populated with questions, and ready to be published.

Flow of Events	
Actor action	System Response
1. The teacher clicks "Create New" and enters the exam title, description, duration, and allowed attempts. 2. The teacher clicks "Create & Continue." 3. The teacher adds questions to the exam, either newly created or reused from the Question Bank. 4. The teacher clicks "Save & Continue."	1. The system validates the fields and automatically assigns the exam a system-generated exam code. 2. The system saves the exam as a draft and redirects the teacher to the Exam Builder. 3. The system attaches the selected questions to the exam and tracks unsaved changes. 4. The system persists the exam's question list.
Alternative Scenarios	
Actor action	System Response
1a. The teacher leaves the title blank or sets a duration of zero. 2a. The teacher clicks "Cancel." 3a. The teacher tries to leave the Exam Builder with unsaved question edits.	1a. The system disables the "Create & Continue" button until the missing fields are corrected. 2a. The system discards the entered details and closes the modal without creating an exam. 3a. The system prompts the teacher to save or discard changes before navigating away.

Table 10. Create New Exam

Use Case 03: Manage Question Bank

Use Case	Manage Question Bank

Actors	Teacher
Includes	Create Coding Question, Create MCQ
Pre-condition	The teacher is logged in and navigates to the Question Bank.
Post-Condition	The teacher's pool of reusable questions is updated.
Flow of Events	
Actor action	System Response
1. The teacher opens the Question Bank and clicks "New Question." 2. The teacher searches or filters existing questions by tag or type. 3. The teacher deletes a question from the bank.	1. The system prompts the teacher to choose a question type from either coding or multiple choice, triggering either the Create Coding Question or Create MCQ use case. 2. The system filters the displayed list accordingly. 3. The system removes the question after confirmation.
Alternative Scenarios	
Actor action	System Response
1a. The teacher deletes a question from the bank. 2a. The teacher creates or deletes a tag. 3a. The teacher tries to leave with unsaved edits. 4a. The question bank is empty.	1a. The system removes the question from the bank after confirmation. 2a. The system updates the tag list and reflects the change across all questions using that tag. 3a. The system prompts the teacher to confirm

	before discarding the changes. 4a. The system displays a message prompting the teacher to create their first question.

Table 11. Manage Question Bank

Use Case 04: Create Coding Question

Use Case	Create Coding Question
Actors	Teacher
Included in	Manage Question Bank
Pre-condition	The teacher has selected "Coding Question" as the question type from the Question Bank or Exam Builder's Question Editor.
Post-Condition	A new coding question, complete with test cases, is saved to the Question Bank.
Flow of Events	
Actor action	System Response
1. The teacher enters the question prompt, starter code, and main code, and selects a program type. 2. The teacher adds one or more test cases, each with an input, expected output, and a hidden/visible flag. 3. The teacher sets a check-penalty-per-attempt value and, optionally, a reference solution. 4. The teacher clicks "Save Question."	1. The system displays a code-editor panel pre-filled with the starter and main code templates. 2. The system lists each test case under the question. 3. The system validates that required fields are filled before enabling Save. 4. The system stores the question and its test cases in the Question Bank.

Alternative Scenarios	
Actor action	System Response
1a. The teacher leaves the prompt or all test cases blank. 2a. The teacher cancels before saving.	1a. The system keeps the Save button disabled until the missing fields are completed. 2a. The system discards the entered question and returns to the Question Bank list.

Table 12. Create Coding Question

Use Case 05: Create MCQ

Use Case	Create MCQ
Actors	Teacher
Included in	Manage Question Bank
Pre-condition	The teacher has selected "Multiple-choice Question" as the question type from the Question Bank or Exam Builder's Question Editor.
Post-Condition	A new multiple-choice question, complete with test cases, is saved to the Question Bank.
Flow of Events	
Actor action	System Response
1. The teacher enters the question prompt. 2. The teacher adds choices and marks one as correct. 3. The teacher clicks "Save."	1. The system displays an editor for the question text and its answer choices. 2. The system tracks each option's text and correctness flag. 3. The system validates that one option is

	marked correct, then stores the question in the Question Bank.
Alternative Scenarios	
Actor action	System Response
1a. The teacher saves without marking any option as correct. 2a. The teacher cancels before saving.	1a. The system displays a validation error and blocks saving. 2a. The system discards the entered question and returns to the Question Bank list.

Table 13. Create MCQ

Use Case 06: Publish Exam

Use Case	Publish Exam
Actors	Teacher
Includes	Generate Exam Code, Create Verification Code
Pre-condition	The teacher has a draft or saved exam with at least one question in the Exam Library.
Post-Condition	The exam status changes to "active," and students may join using the verification code.
Flow of Events	
Actor action	System Response
1. The teacher selects "Publish Exam" from an exam card's menu. 2. The teacher confirms publishing.	1. The system checks whether the exam already has a verification code set, triggering the Create Verification Code

	use case if not. 2. The system sets the exam status to "active." 3. The system displays a success overlay confirming the exam code, showing the Generate Exam Code use case's output.
Alternative Scenarios	
Actor action	System Response
1a. Publishing fails due to a system or network error.	1a. The system displays an error inside the modal and keeps the exam in its current status.

Table 14. Publish Exam

Use Case 07: Generate Exam Code

Use Case	Generate Exam Code
Actors	Teacher
Included in	Publish Exam
Pre-condition	The teacher is publishing an exam.
Post-Condition	The exam's unique exam code is confirmed and displayed to the teacher.
Flow of Events	
Actor action	System Response
1. The teacher completes the Publish Exam	1. The system retrieves the exam code

action.	that was generated automatically when the exam was created. 2. The system displays the exam code in the publish success overlay for the teacher to share or record.
Alternative Scenarios	
Actor action	System Response
1a. The exam code fails to load.	1a. The system displays a fallback message and allows the teacher to retrieve the code later from the Exam Library.

Table 15. Generate Exam Code

Use Case 08: Create Verification Code

Use Case	Create Verification Code
Actors	Teacher
Included in	Publish Exam
Pre-condition	The teacher is publishing an exam that does not yet have a verification code.
Post-Condition	A 6-digit verification code is attached to the exam for students to join with.
Flow of Events	
Actor action	System Response
1. The teacher navigates to "Exam Settings" or the system prompts the	1. The system displays an input field accepting numeric digits only.

teacher to set a session verification code. 2. The teacher enters a 6-digit code. 3. The teacher confirms.	2. The system validates that the code is exactly six digits. 3. The system stores the verification code and proceeds to publish the exam.
Alternative Scenarios	
Actor action	System Response
1a. The teacher enters fewer than 6 digits or non-numeric characters. The teacher tries to edit the verification code after the exam is already active.	1a. The system strips invalid characters and keeps the confirm button disabled until the code is valid. 2a. The system displays the code as locked, since it can no longer be changed once published.

Table 16. Create Verification Code

Use Case 09: Monitor Exam Session

Use Case	Monitor Exam Session
Actors	Teacher
Includes	View Violations
Pre-condition	The teacher has an "active" exam in progress.
Post-Condition	The teacher views real-time student session statuses.
Flow of Events	
Actor action	System Response

1. The teacher clicks "View Progress" on an active exam's card. 2. The teacher selects a student's session to inspect further.	1. The system displays each student's session status from either In Progress, Submitted, or Timed Out. 2. The system triggers the View Violations use case for that session.
Alternative Scenarios	
Actor action	System Response
1a. No students have joined the session yet.	1a. The system displays a message indicating no active sessions.

Table 17. Monitor Live Exam Session

Use Case 10: View Violations

Use Case	View Violations
Actors	Teacher
Included in	Monitor Exam Session
Pre-condition	The teacher has selected a student's session while monitoring an active exam.
Post-Condition	The teacher views the list of violations logged for that session.
Flow of Events	
Actor action	System Response
1. The teacher clicks on a student's session row. 2. The teacher closes the modal.	1. The system opens a modal listing logged violations, such as Alt+Tab, Windows Key, Screenshot, or

	Copy/Paste attempts. 2. The system returns to the session overview.
Alternative Scenarios	
Actor action	System Response
1a. The session has no violations logged.	1a. The system displays a message stating no violations were recorded.

Table 18. View Violations

Use Case 11: End Exam

Use Case	End Exam
Actors	Teacher
Pre-condition	The teacher has an "active" exam.
Post-Condition	The exam status changes to "ended," and students can no longer join or submit answers.
Flow of Events	
Actor action	System Response
1. The teacher selects "End Exam" from an active exam card's menu. 2. The teacher confirms action.	1. The system displays a confirmation dialog warning that students will no longer be able to join or submit answers. 2. The system marks the exam as "ended" and refreshes the Exam Library.

Alternative Scenarios	
Actor action	System Response
1a. The teacher cancels the confirmation dialog.	1a. The system closes the dialog and leaves the exam active.

Table 19. End Exam

Use Case 12: Review Exam Results

Use Case	Review Exam Results
Actors	Teacher
Pre-condition	The teacher has at least one exam with "ended" status.
Post-Condition	The teacher views the class performance summary and, optionally, downloads the results.
Flow of Events	
Actor action	System Response
1. The teacher selects an “ended” exam from Exam Results. 2. The teacher searches for a student by name. 3. The teacher clicks "View Answers" on a student's session. 4. The teacher clicks "Export CSV" or "Export PDF."	1. The system displays class average, pass rate, and highest score for that exam. 2. The system filters the results table to matching sessions. 3. The system displays the per-question correctness and score for that attempt. 4. The system generates and downloads the results file.

Alternative Scenarios	
Actor action	System Response
1a. A student attempted the exam more than once. 2a. No exams have ended yet.	1a. The system displays each attempt as its own row, labeled with its attempt number. 2a. The system displays a message indicating no exams have ended.

Table 20. View and Export Exam Results

Use Case Narratives (Admin)

Use Case 01: Admin Login

Use Case	Admin Login
Actors	Admin
Pre-condition	The admin has been issued an ID number and password by the system. Contx does not support self-registration.
Post-Condition	The admin is authenticated and redirected to the Admin Dashboard.
Flow of Events	
Actor action	System Response
1. The admin opens the Contx web portal and enters their ID number and password. 2. The admin submits the login form.	1. The system displays the login form requesting an ID Number and Password. 2. The system looks up the account by ID number, authenticates the

	credentials, and retrieves the admin's profile. 3. The system redirects the admin to the Admin Dashboard.
Alternative Scenarios	
Actor action	System Response
1a. The admin enters an invalid ID number or password.	1a. The system displays a generic "Invalid ID number or password" message without specifying which field is incorrect.

Table 21. Admin Login

Use Case 02: Create Account Manually

Use Case	Create Account Manually
Actors	Admin
Pre-condition	The admin is logged in and viewing the User Management page.
Post-Condition	A new account is created with the default password "temp123" and appears in the account list.
Flow of Events	
Actor action	System Response
1. The admin clicks "Add Account" and fills in Student ID, First Name, Last Name, Gmail, and role (Student or Teacher).	1. The system displays a form requesting the account details. 2. The system validates that all required fields are filled and that the Student ID

2. The admin clicks "Create."	is not already in use. 3. The system creates the account with "temp123" as the default password and adds it to the account list.
Alternative Scenarios	
Actor action	System Response
1a. The admin leaves a required field blank. 2a. The admin enters a Student ID that already exists.	1a. The system displays a generic "There is an Empty field" message without specifying which field is blank 2. The system displays an error message indicating the ID is already in use and blocks account creation.

Table 22. Create Account Manually

Use Case 03: Create Account via CSV Upload

Use Case	Create Account via CSV Upload
Actors	Admin
Pre-condition	The admin is logged in and viewing the User Management page. The admin has a CSV file containing Student ID, First Name, Last Name, and Email.
Post-Condition	New accounts are created with the default password "temp123", and the accounts appear in the account list.

Flow of Events	
Actor action	System Response
1. The admin clicks "Upload CSV" and selects the file. 2. The admin confirms the upload.	1. The system displays a file upload dialog. 2. The system parses the CSV and validates that each row contains a Student ID, First Name, Last Name, and Gmail. 3. The system creates an account for each valid row, assigning "temp123" as the default password. 4. The system displays a summary of accounts created and any rows that failed validation.
Alternative Scenarios	
Actor action	System Response
1a. The CSV file is missing a required column or contains a malformed row. 2a. The CSV contains a Student ID that already exists in the system.	1a. The system skips the invalid row, continues processing the rest of the file, and lists the skipped rows in the summary. 2. The system skips the duplicate entry and flags it in the summary instead of creating a second account.

Table 23. Create Account via CSV Upload

Use Case 04: Update Account

Use Case	Create Account via CSV Upload
Actors	Admin

Pre-condition	The admin is logged in and viewing the User Management page. At least one account exists.
Post-Condition	The account's information is updated and reflected in the account list.
Flow of Events	
Actor action	System Response
1. The admin selects an account and clicks "Edit." 2. The admin updates the desired fields and clicks "Save."	1. The system displays the account's current details in an editable form. 2. The system validates the updated fields and saves the changes. 3. The system displays a confirmation and refreshes the account list.
Alternative Scenarios	
Actor action	System Response
1a. The admin clears a required field before saving.	1a. The system displays a generic "There is an Empty field" message without specifying which field is blank

Table 24. Update Account

Use Case 05: Delete Account

Use Case	Create Account via CSV Upload
Actors	Admin
Pre-condition	The admin is logged in and viewing the User Management page. At least one account

	exists.
Post-Condition	The account is removed from the active account list and can no longer log in.
Flow of Events	
Actor action	System Response
1. The admin selects an account and clicks "Delete." 2. The admin confirms the deletion.	1. The system displays a confirmation prompt warning that the action will remove the account's access. 2. The system deletes the account and removes it from the active account list.
Alternative Scenarios	
Actor action	System Response
1a. The admin clicks "Cancel" on the confirmation prompt.	1a. The system closes the prompt and takes no action.

Table 25. Delete Account

Activity Diagram (Student)

Student Login

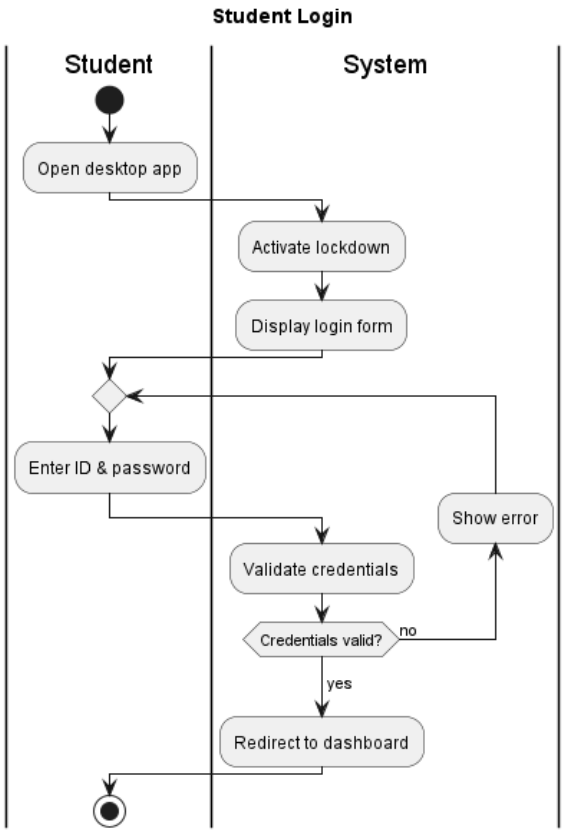


Figure 3.1 - Student Login

Figure 3.1 shows the process for student login on the Cont/x platform. It starts with the student opening the desktop application, after which the system activates lockdown and displays the login form requesting an ID number and password. The student then enters their credentials and submits them for validation. If the credentials are invalid, the system displays an error message and returns the student to the login form to try again. This repeats until the student enters valid credentials. Once validation succeeds, the system redirects the student to the Dashboard. This ensures that only authenticated students can access the exam-taking environment, with lockdown already in place before any further interaction occurs.

Take Exam

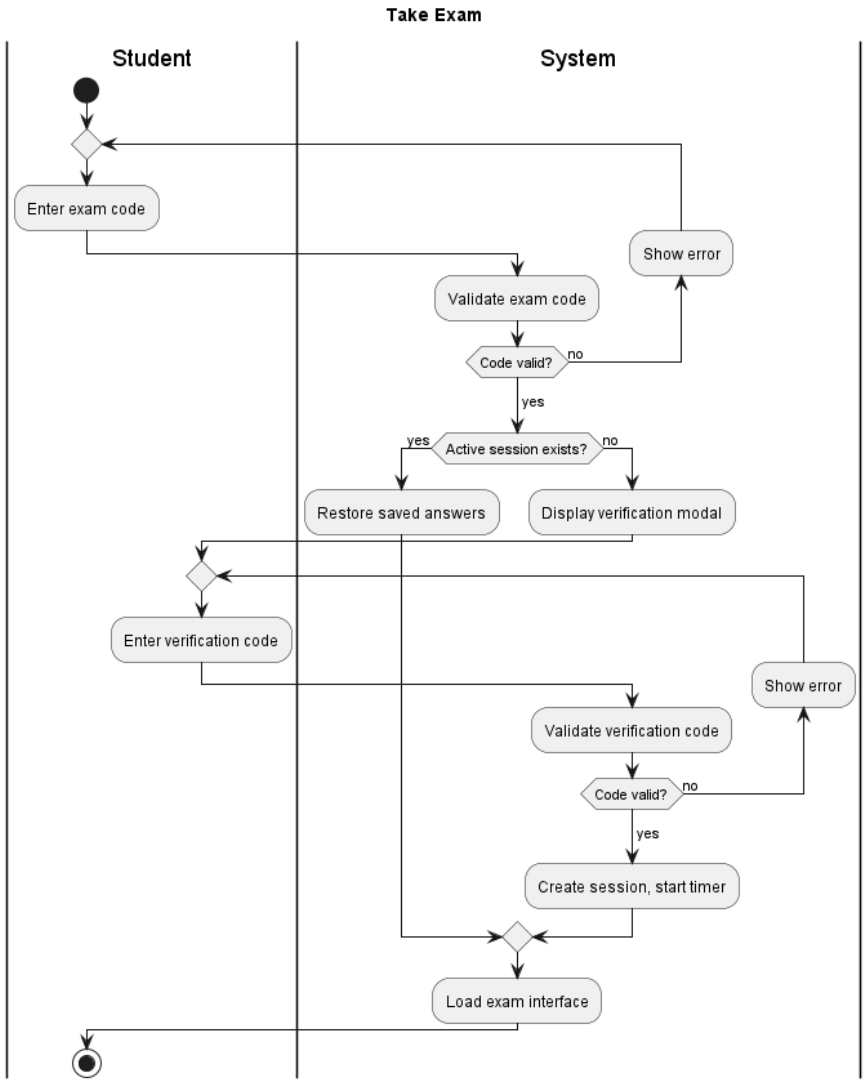


Figure 3.2 - Take Exam

Figure 3.2 shows the process for taking an exam on the Cont/x platform. It starts with the student entering an exam code, which the system validates. If the code is invalid, the system displays an error message and prompts the student to re-enter it, repeating until a valid code is submitted. Once the exam code is validated, the system checks whether an in-progress session already exists for that student on that exam. If a session already exists, the system restores the student's previously saved answers, skipping the verification step. If no session exists, the system displays a verification modal prompting the student for a verification code. The student enters the code, which the system validates; if invalid, an error is shown and the student is prompted to

try again, repeating until the correct code is entered. Once verified, the system creates the exam session and starts the timer. In both paths, the system then loads the exam interface for the student. This ensures that returning students can resume interrupted exams seamlessly, while new sessions are properly verified before access is granted.

Answer Question

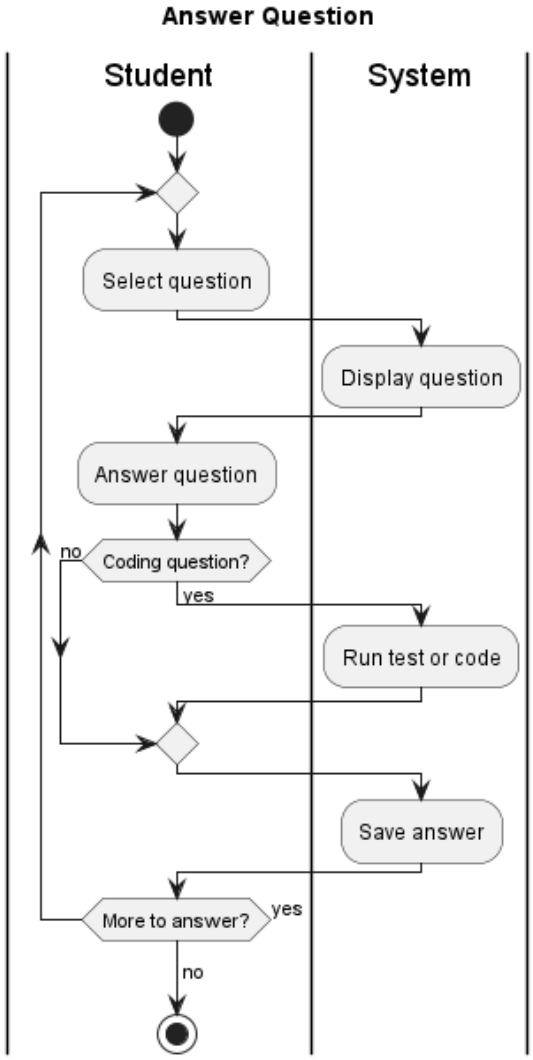


Figure 3.3 - Answer Question

Figure 3.3 shows the process for answering questions during an exam on the Cont/x platform. It starts with the student selecting a question from the question navigator, after which the system displays the question prompt along with its type-specific input. The student then

provides an answer, whether by selecting an option, typing a response, or writing code. If the question is a coding question, the student may run their code or test cases, with the system executing and displaying the results before the student finalizes their answer. Once the student is done with the question, the system saves the answer against the session. This process repeats as the student moves between questions until all questions have been addressed, at which point the activity ends. This ensures that answers are saved continuously as the student progresses through the exam, regardless of question type.

Run Test

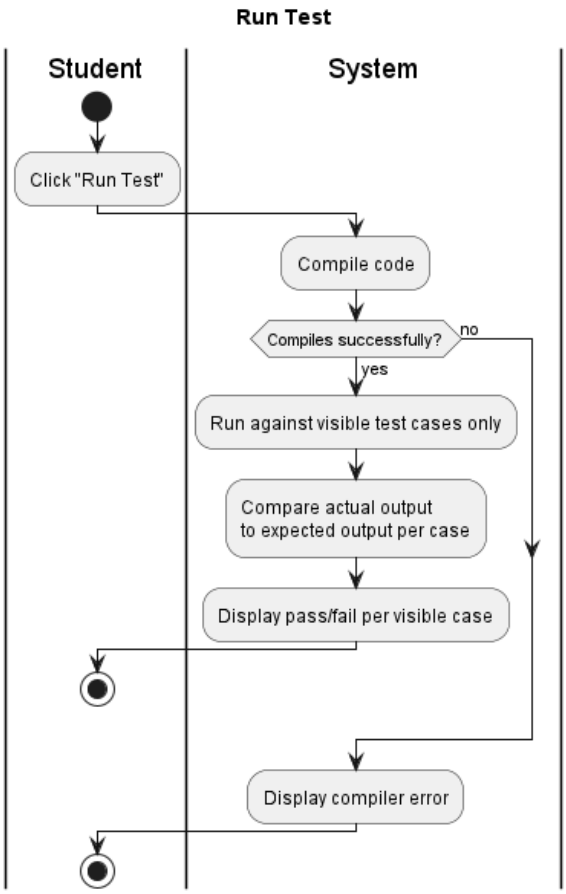


Figure 3.4 - Run Test

Figure 3.4 shows the process for running a test on the Cont/x platform. It starts with the student clicking "Run Test," after which the system compiles the student's code. If the code compiles successfully, the system executes it against the visible test cases only, compares each actual output to the expected output, and displays the pass/fail result for each visible case to the

student. If the code fails to compile, the system instead displays the compiler error output to the student, without running any test cases. This ensures that students can check their code against sample cases for practice without it counting toward their check attempts or point deductions.

Run Code

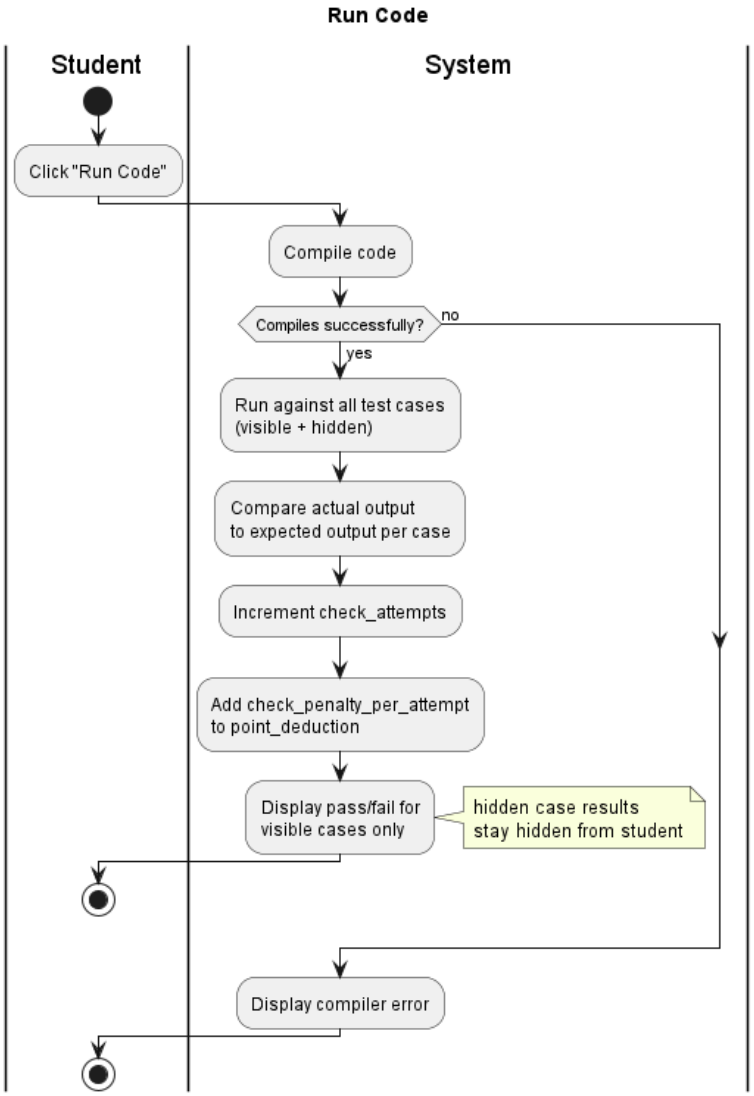


Figure 3.5 - Run Code

Figure 3.5 shows the process for running code on the Cont/x platform. It starts with the student clicking "Run Code," after which the system compiles the student's code. If the code compiles successfully, the system executes it against all test cases, both visible and hidden, and compares each actual output to the expected output. The system then increments the student's check attempt count for that question and adds the configured penalty per attempt to the point

deduction. The system displays the pass/fail results for the visible test cases only, keeping hidden case results concealed from the student. If the code fails to compile, the system instead displays the compiler error output, without running any test cases or incrementing the check attempt count. This ensures that students receive immediate feedback on their code while a running penalty is tracked for repeated check attempts against hidden test cases.

Submit Exam

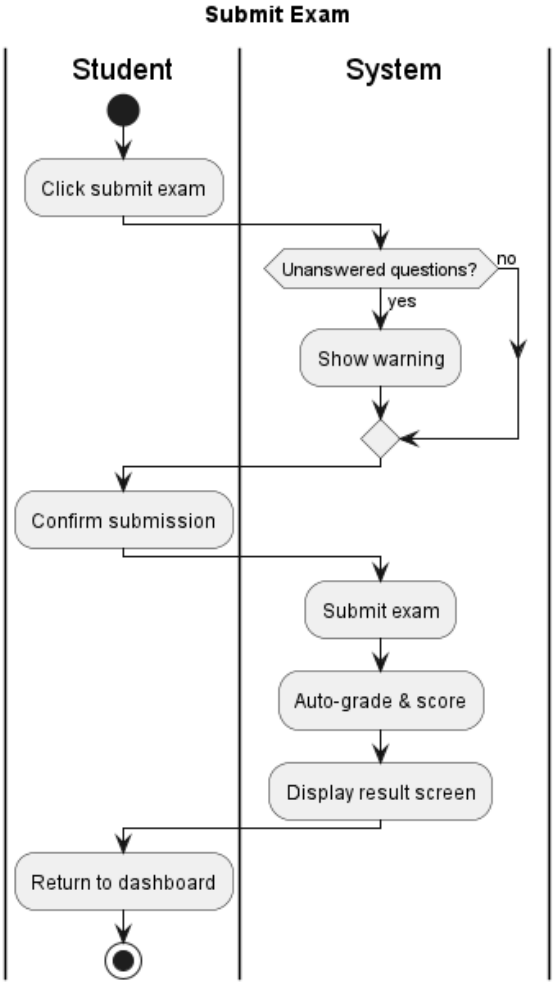


Figure 3.6 - Submit Exam

Figure 3.6 shows the process for submitting an exam on the Cont/x platform. It starts with the student clicking "Submit Exam." The system then checks whether any questions remain unanswered; if so, it displays a warning before proceeding. The student confirms the submission, and the system marks the exam session as submitted, recording the timestamp. The system then

auto-grades the coding, MCQ, and fill-in-the-blank answers, applying any accumulated point deductions, and computes the total score. The system displays the Exam Result Screen, showing the time taken, the student's score, and the number of correctly answered questions. Finally, the student clicks the button to return to the Dashboard. This ensures that students are warned about incomplete answers before final submission, while scoring and results are handled automatically once the exam is submitted.

Activity Diagram (Teacher)

Manage Question Bank

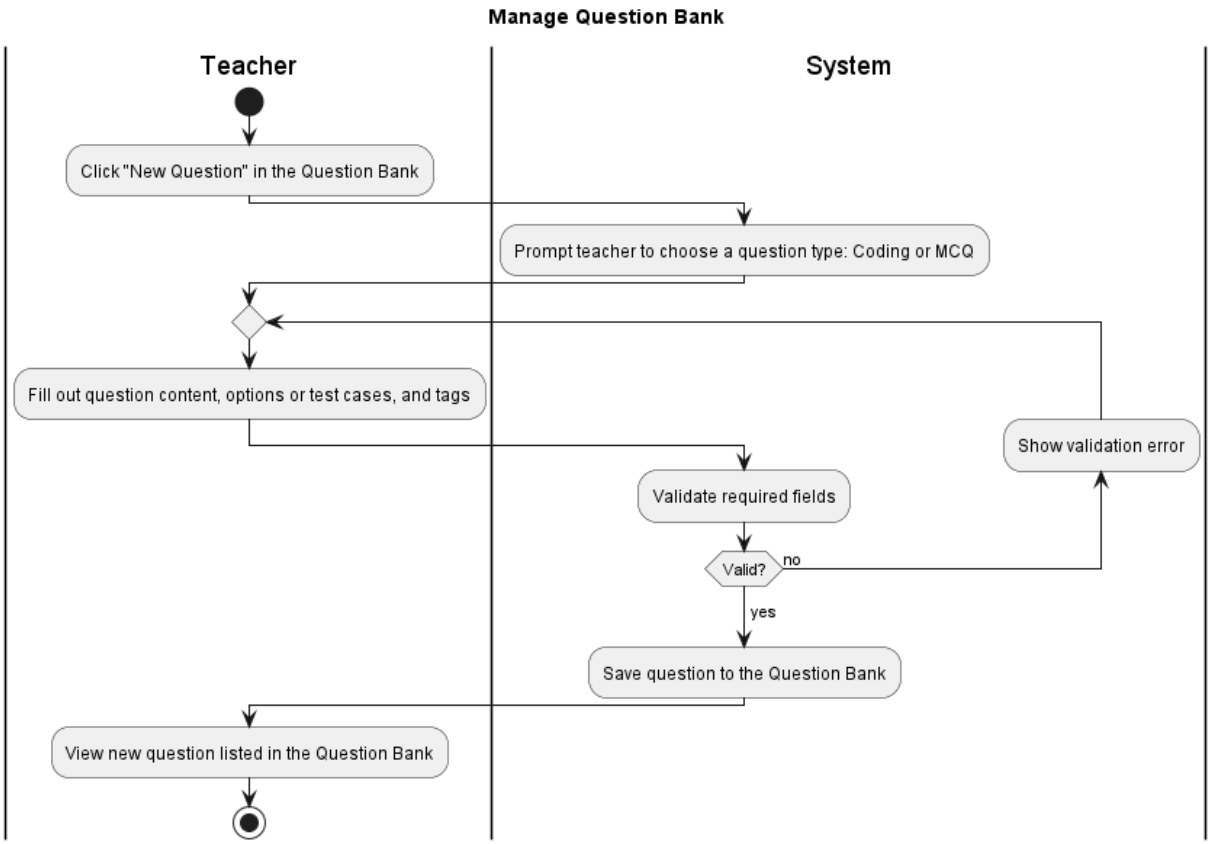


Figure 3.7 - Manage Question Bank

Figure 3.7 shows the process of managing the Question Bank on the Cont/x web application. The process begins when the teacher clicks "New Question," prompting the system to ask whether the question is a coding or multiple-choice type. The teacher fills out the question content, options or test cases, and relevant tags, which the system validates for completeness. If

any required field is missing or invalid, the system displays an error and the teacher is returned to the form to make corrections. Once all fields are valid, the system saves the question to the teacher's Question Bank and displays it in the list, confirming that the question is ready to be reused across exams.

Create Exam

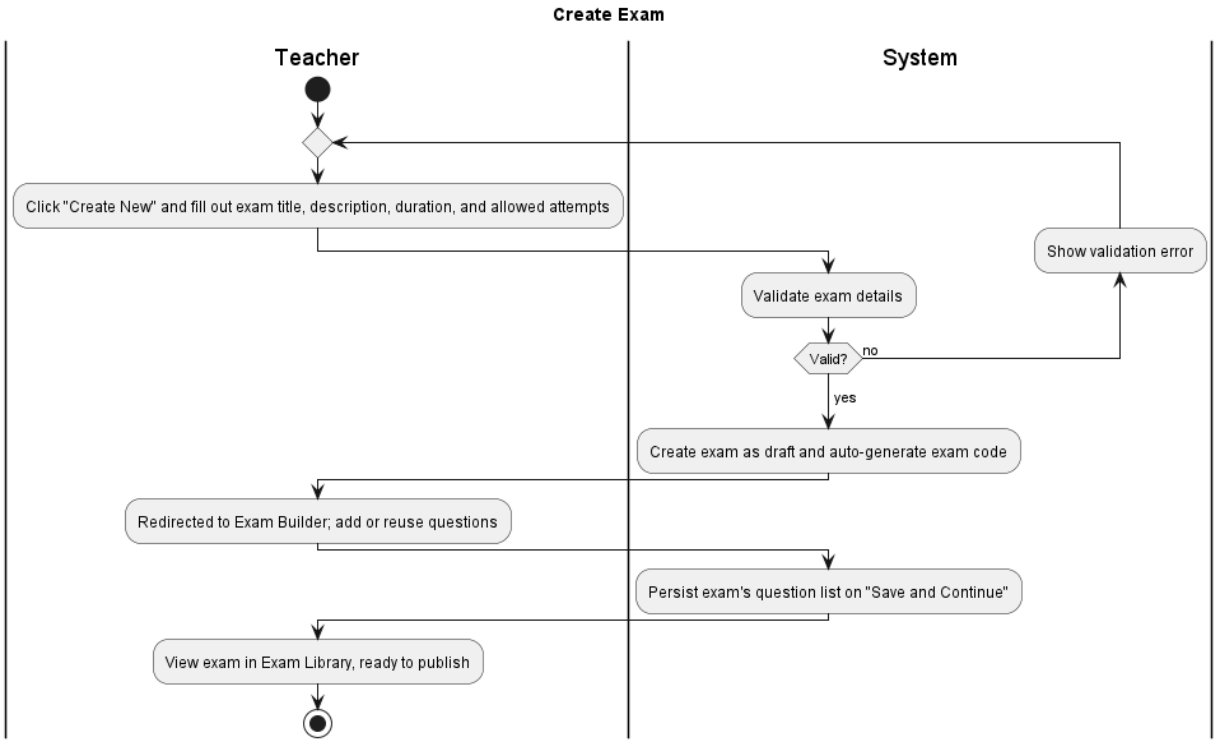


Figure 3.8 - Create Exam

Figure 3.8 illustrates the exam creation process. The teacher fills out the exam title, description, duration, and allowed attempts, which the system validates. If the details are incomplete, the system prompts the teacher to correct them before proceeding. Once valid, the system creates the exam as a draft and automatically generates its unique exam code. The teacher is then redirected to the Exam Builder to add or reuse questions, and once saved, the exam appears in the Exam Library, ready to be published.

Publish Exam

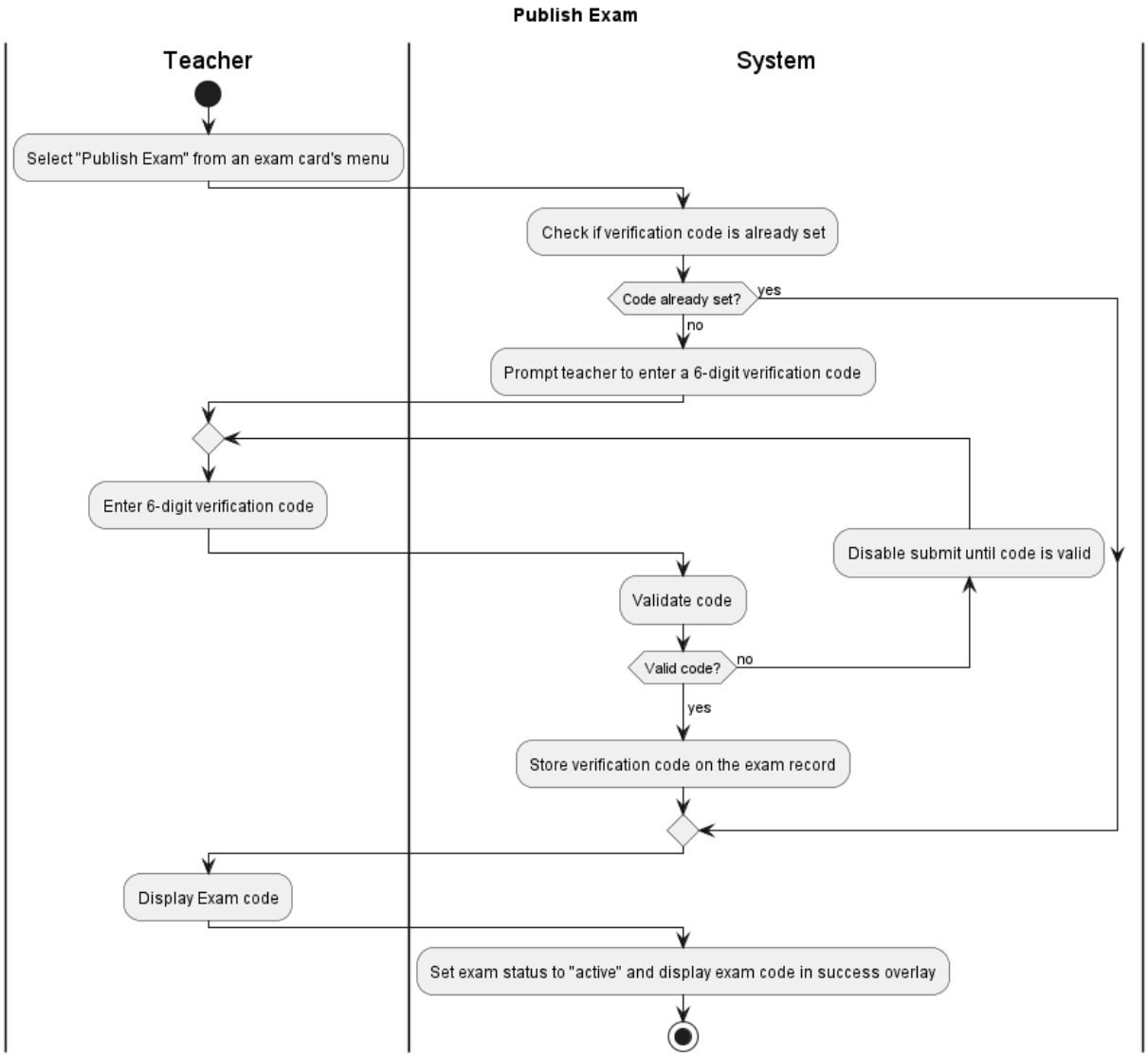


Figure 3.9 - Publish Exam

Figure 3.9 details the process of publishing an exam. When the teacher selects "Publish," the system checks whether a verification code has already been set. If not, the teacher is prompted to enter a 6-digit code, which the system validates before storing it on the exam. Once the code is set or if it was already set previously the teacher confirms publishing, and the system changes the exam's status to "active," displaying the exam code in a success overlay for the teacher to share with students.

Monitor Exam Session

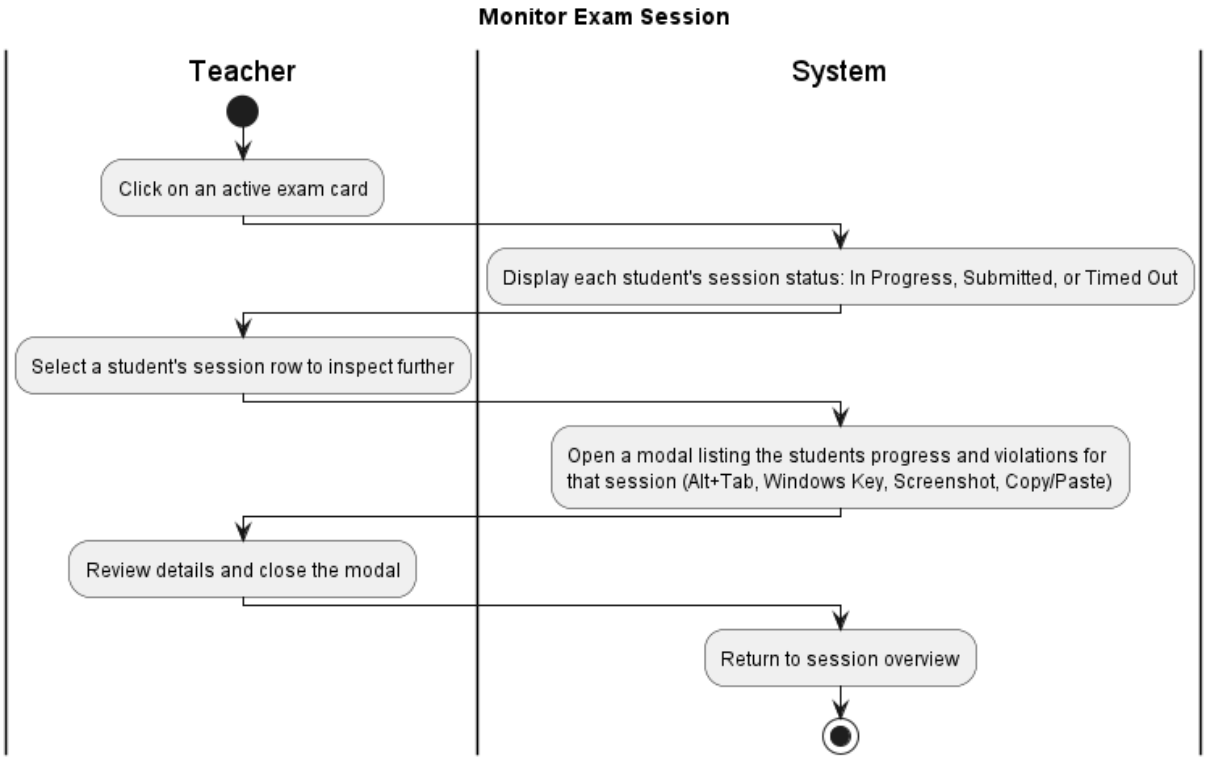


Figure 3.10 - Monitor Exam Session

Figure 3.10 shows how a teacher monitors an active exam session. After clicking "View Progress," the system displays each student's session status as In Progress, Submitted, or Timed Out. Selecting a student's session opens a modal listing any violations logged for that session, such as Alt+Tab, Windows Key, screenshot, or copy/paste attempts. Once the teacher reviews the details and closes the modal, the system returns to the session overview.

End Exam

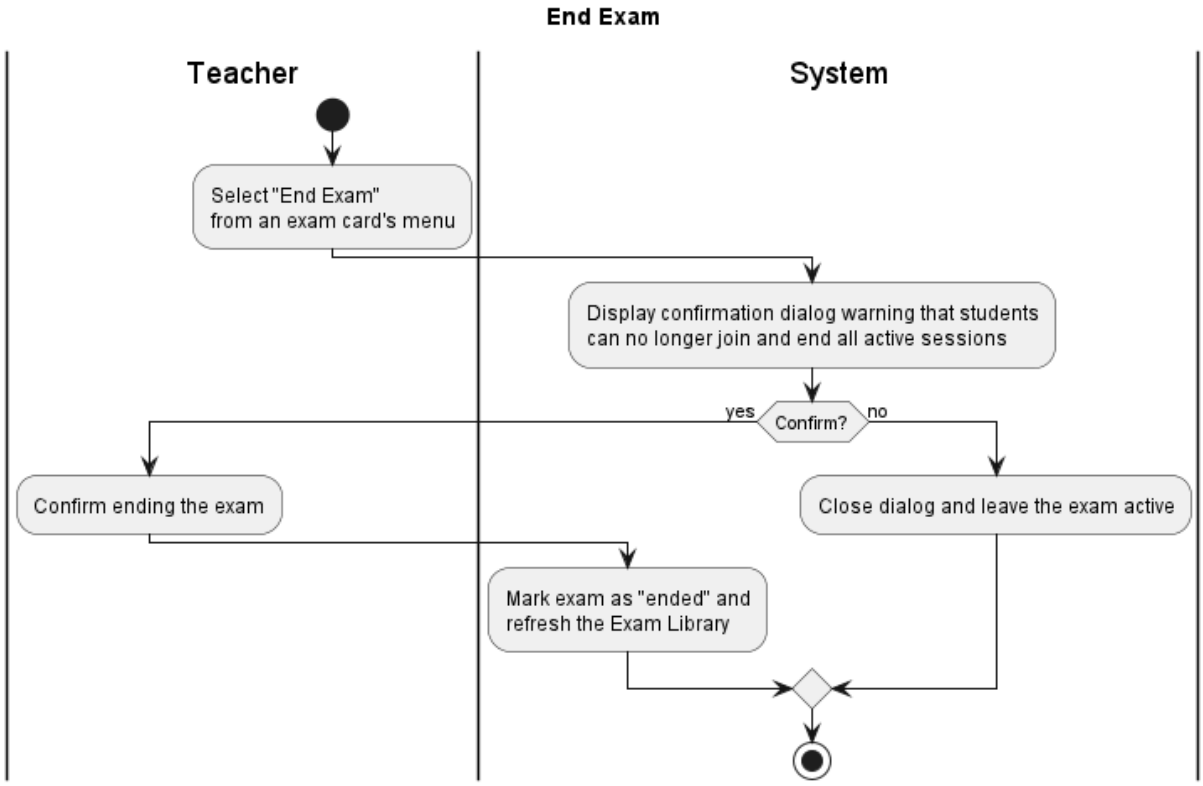


Figure 3.11 - End Exam

Figure 3.11 illustrates the process of ending an active exam. When the teacher selects "End Exam," the system displays a confirmation dialog warning that students will no longer be able to join or submit answers. If the teacher cancels, the dialog closes and the exam remains active. If confirmed, the system marks the exam as "ended" and refreshes the Exam Library, preventing any further student activity on that exam.

Review Exam Result

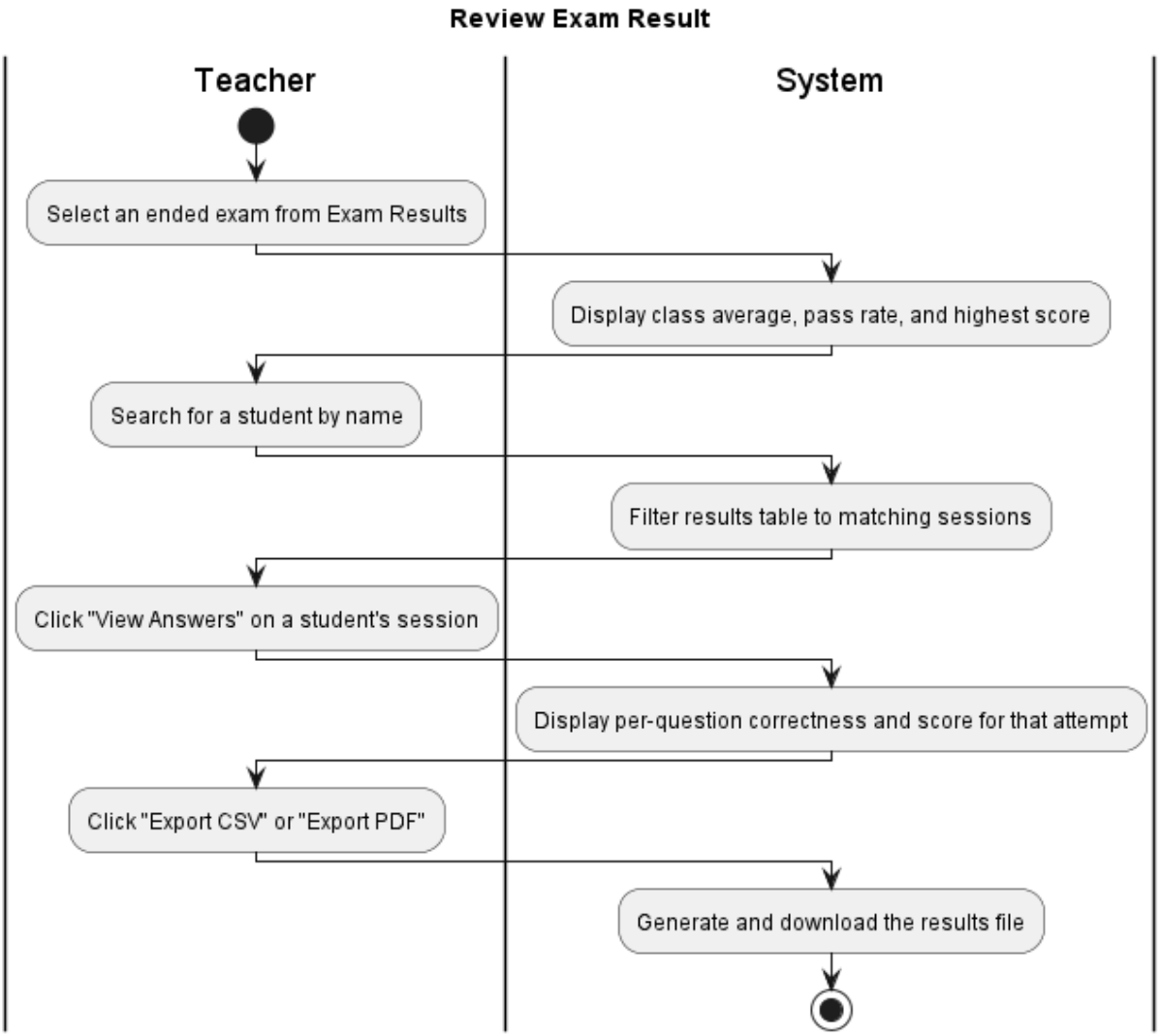


Figure 3.12 - Review Exam Result

Figure 3.12 shows the process of reviewing exam results. After selecting an “ended” exam, the system displays the class average, pass rate, and highest score. The teacher can search for a specific student, which filters the results table, and click "View Answers" to see the per-question correctness and score for that attempt. Finally, the teacher can export the results as a CSV or PDF file, which the system generates and downloads.

Activity Diagram (Admin)

Admin Login

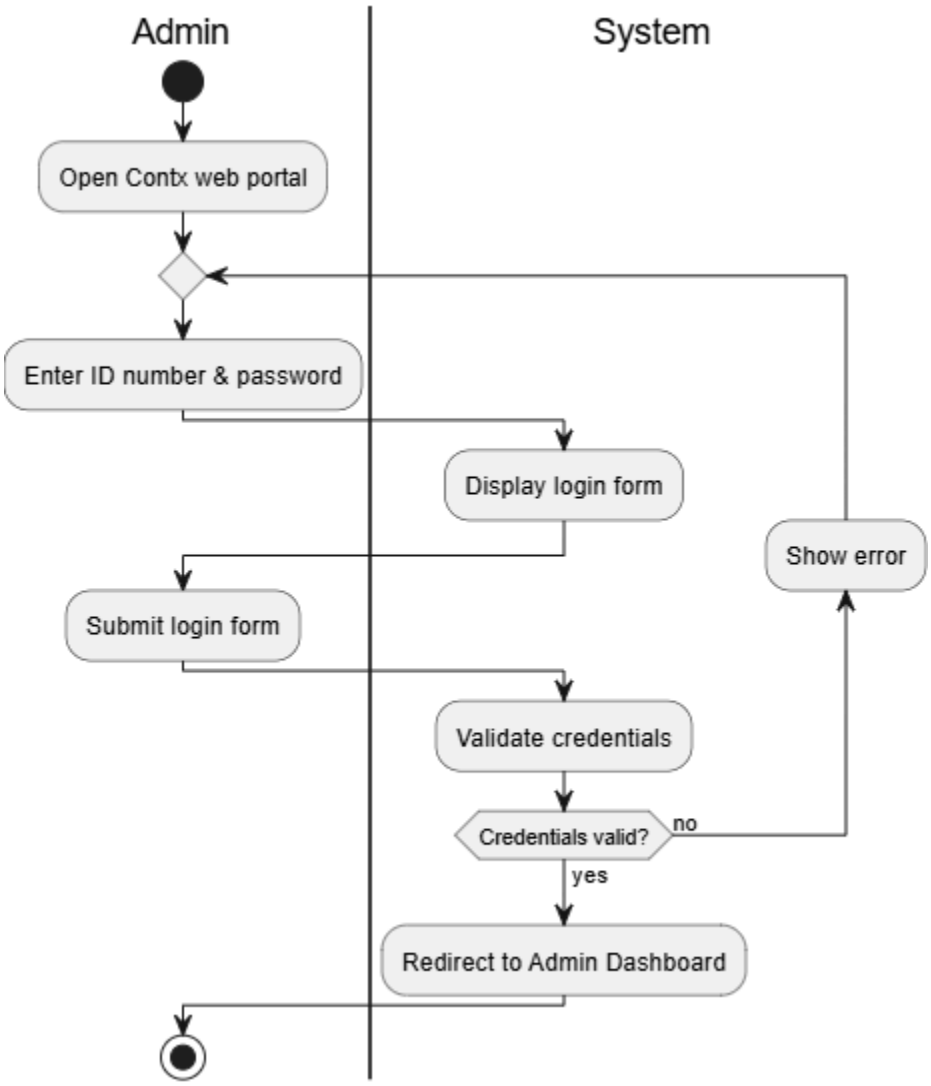


Figure 3.13 - Admin Login

Figure 3.13 shows the process for admin login on the Cont/x platform. It starts with the admin opening the web portal and entering their ID number and password. The system validates the submitted credentials. If the credentials are invalid, the system displays an error message and returns the admin to the login form to try again. This repeats until valid credentials are entered. Once validation succeeds, the system redirects the admin to the Admin Dashboard. This ensures that only authenticated admins can access account management functions.

Create Account Manually

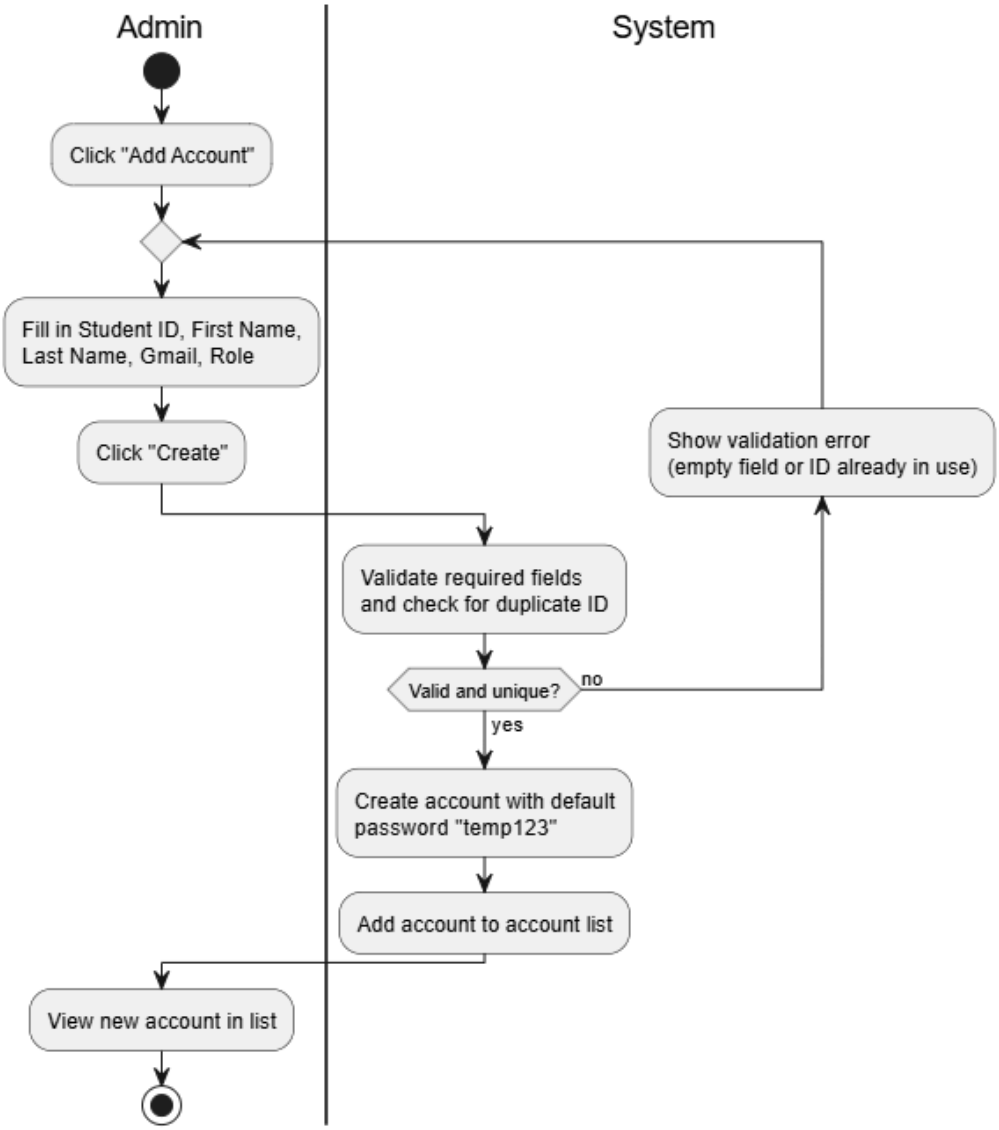


Figure 3.14 - Create Account Manually

Figure 3.14 shows the process for manually creating an account on the Cont/x platform. It starts with the admin clicking "Add Account" and filling in the Student ID, First Name, Last Name, Email, and role. The system validates that all required fields are filled. If a field is left blank, the system displays an error and returns the admin to the form to make corrections. If all fields are filled, the system checks whether the Student ID already exists. If it does, the system displays an error and blocks account creation. If the ID is unique, the system creates the account with the default password "temp123" and adds it to the account list, where the admin can then

view the newly created account. This ensures that every account created manually has a unique identifier and a consistent starting password before it becomes active in the system.

Create Account via CSV Upload

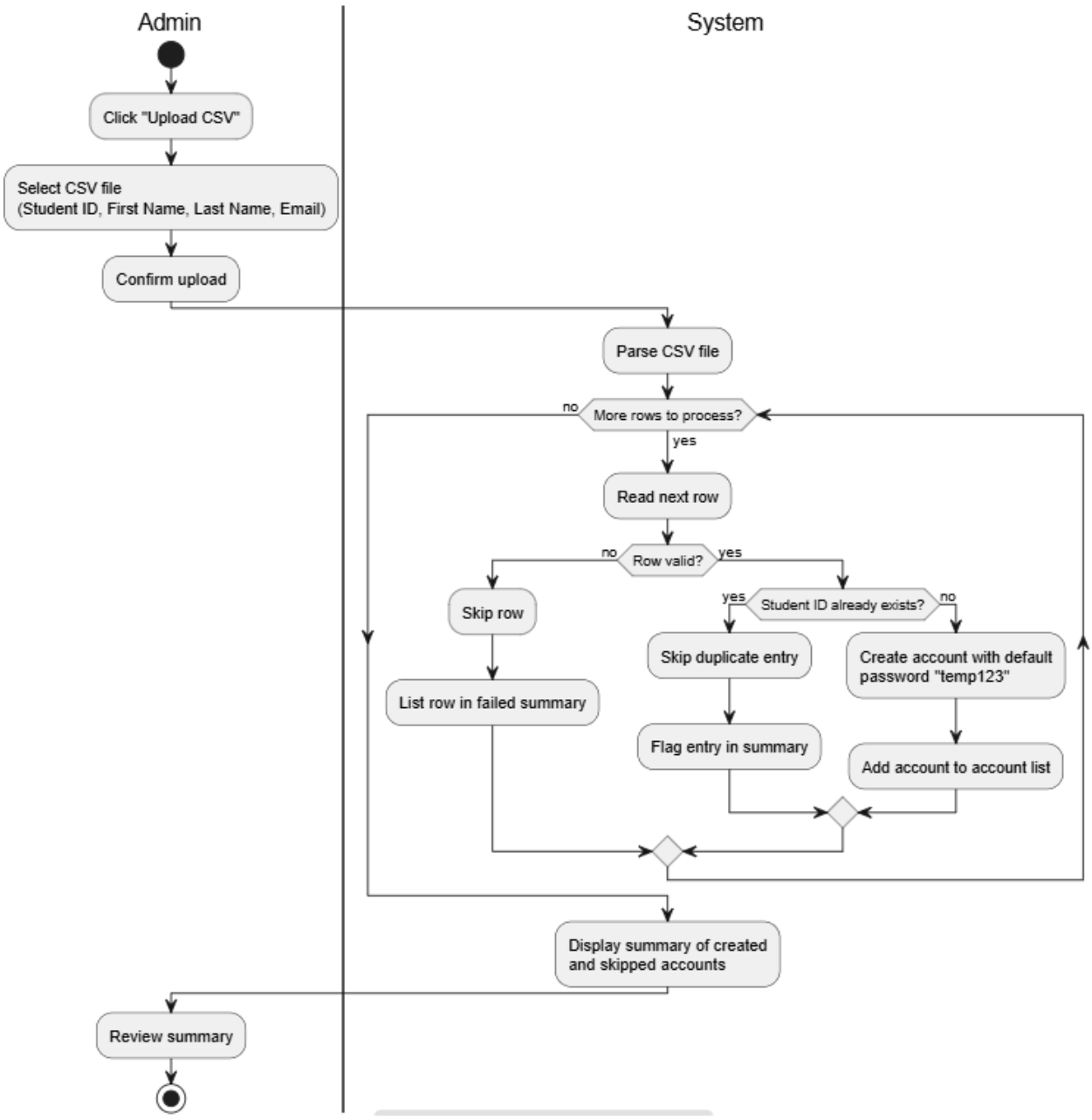


Figure 3.15 - Create Account via CSV Upload

Figure 3.15 shows the process for creating multiple accounts through CSV upload on the Cont/x platform. It starts with the admin selecting a CSV file containing Student ID, First Name, Last Name, and Email, then confirming the upload. The system parses the file and validates each

row individually. If a row is missing a required column or is malformed, the system skips that row and lists it in a failed summary. If a row is valid but its Student ID already exists in the system, the system skips the duplicate and flags it in the summary instead of creating a second account. Valid, non-duplicate rows are created as new accounts with the default password "temp123." This process repeats for every row in the file. Once all rows have been processed, the system displays a summary of the accounts successfully created and any rows that failed. This ensures that bulk account creation can proceed even when a file contains partial errors, without stopping the entire import.

Update Account

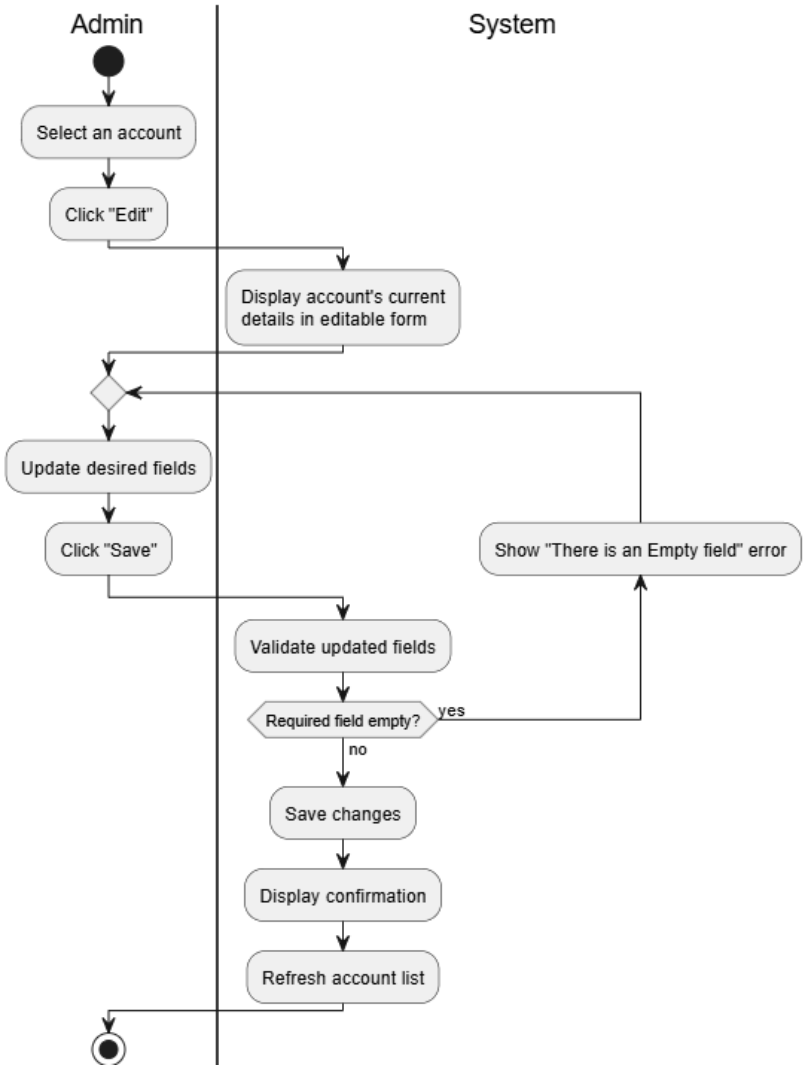


Figure 3.16 - Update Account

Figure 3.16 shows the process for updating an existing account on the Cont/x platform. It starts with the admin selecting an account and clicking "Edit," after which the system displays the account's current details in an editable form. The admin updates the desired fields and clicks "Save." The system validates the updated fields. If a required field is left empty, the system displays an error and returns the admin to the form to make corrections. If all fields are valid, the system saves the changes, displays a confirmation, and refreshes the account list. This ensures that account information stays accurate while preventing incomplete updates from being saved.

Delete Account

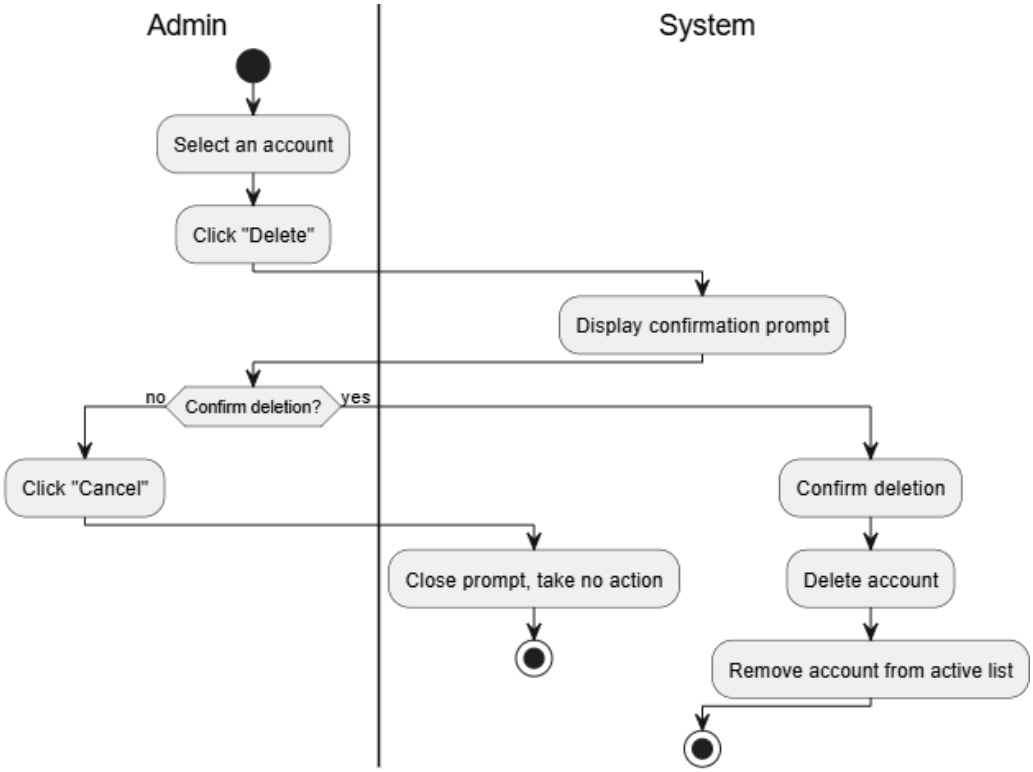


Figure 3.17 - Delete Account

Figure 3.17 shows the process for deleting an account on the Cont/x platform. It starts with the admin selecting an account and clicking "Delete," after which the system displays a confirmation prompt warning that the action will remove the account's access. If the admin cancels, the system closes the prompt and takes no action, leaving the account unchanged. If the admin confirms, the system deletes the account and removes it from the active account list. This ensures that account removal is deliberate and cannot happen accidentally through a single click.

View Audit Logs

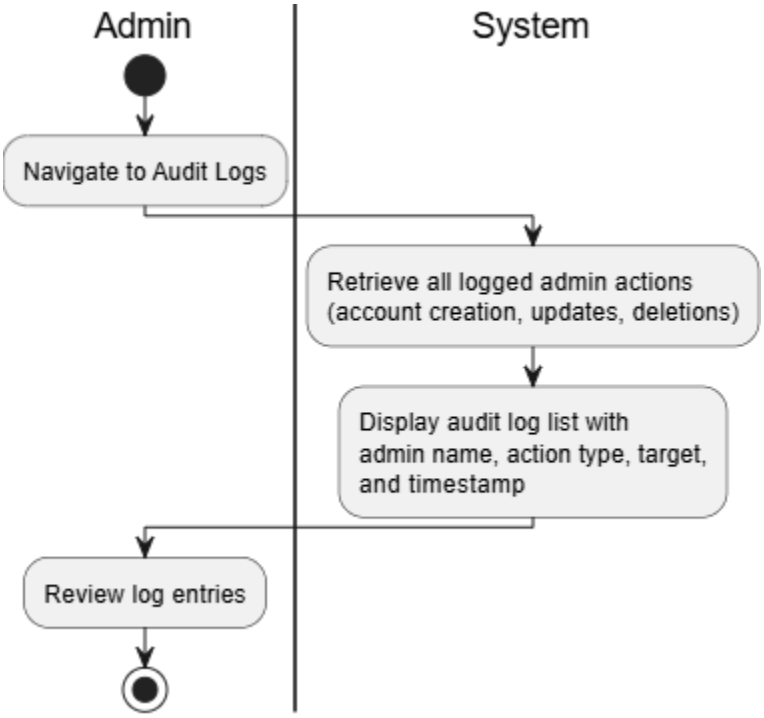


Figure 3.18 - View Audit Logs

Figure 3.18 shows the process for viewing audit logs on the Cont/x platform. It starts with the admin navigating to the Audit Logs section, after which the system retrieves all logged admin actions, including account creation, updates, and deletions. The system then displays the list of log entries, each showing the admin's name, the action type, the affected account, and a timestamp. The admin reviews the entries as needed. This ensures that all account management activity remains traceable and accountable, giving a clear record of who made changes to which accounts and when.

Entity Relationship Diagram

The Entity Relationship Diagram (ERD) depicts the relationship in the database, including attributes with their corresponding primary and foreign keys.

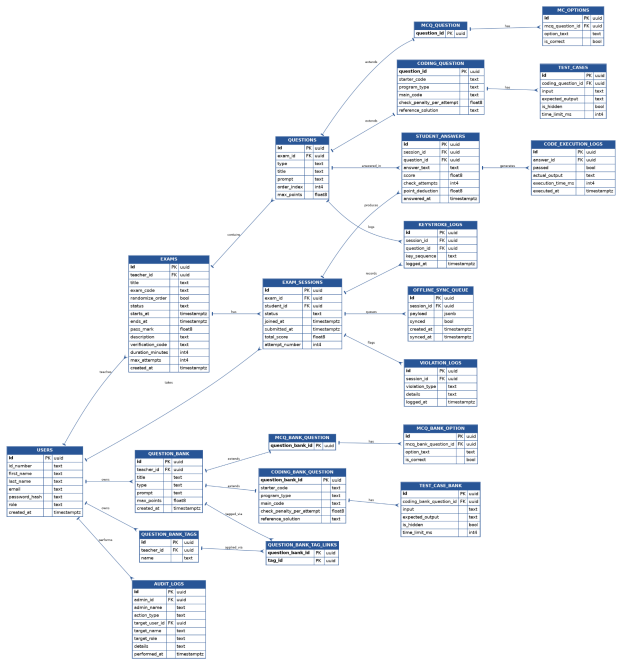


Figure 4.1 - Entity Relationship Diagram

Figure 4.1 shows the Entity Relationship Diagram (ERD) illustrating the core data structure of the Contx system. It includes key entities such as Users, Exams, Questions, Exam Sessions, Student Answers, Question Bank, and Audit Logs. Each entity stores relevant information and connects through defined relationships. Teachers own exams and manage their own question bank, while students take exam sessions and produce answers tied to each question. Questions extend into either MCQ Questions or Coding Questions, each carrying their own options or test cases respectively, allowing the system to grade multiple question types under one structure. Exam sessions generate keystroke logs, violation logs, and an offline sync queue, capturing student activity and lockdown events even when connectivity is unstable. Admins perform actions that are recorded in the audit log, keeping account management traceable. The diagram captures how exam data, grading data, and security data flow together

within a single system, supporting both the lockdown and the automated assessment functions of Contx.

Class Diagram

The class diagram shows the system's structure as classes with their attributes, operations, and relationships among other classes.

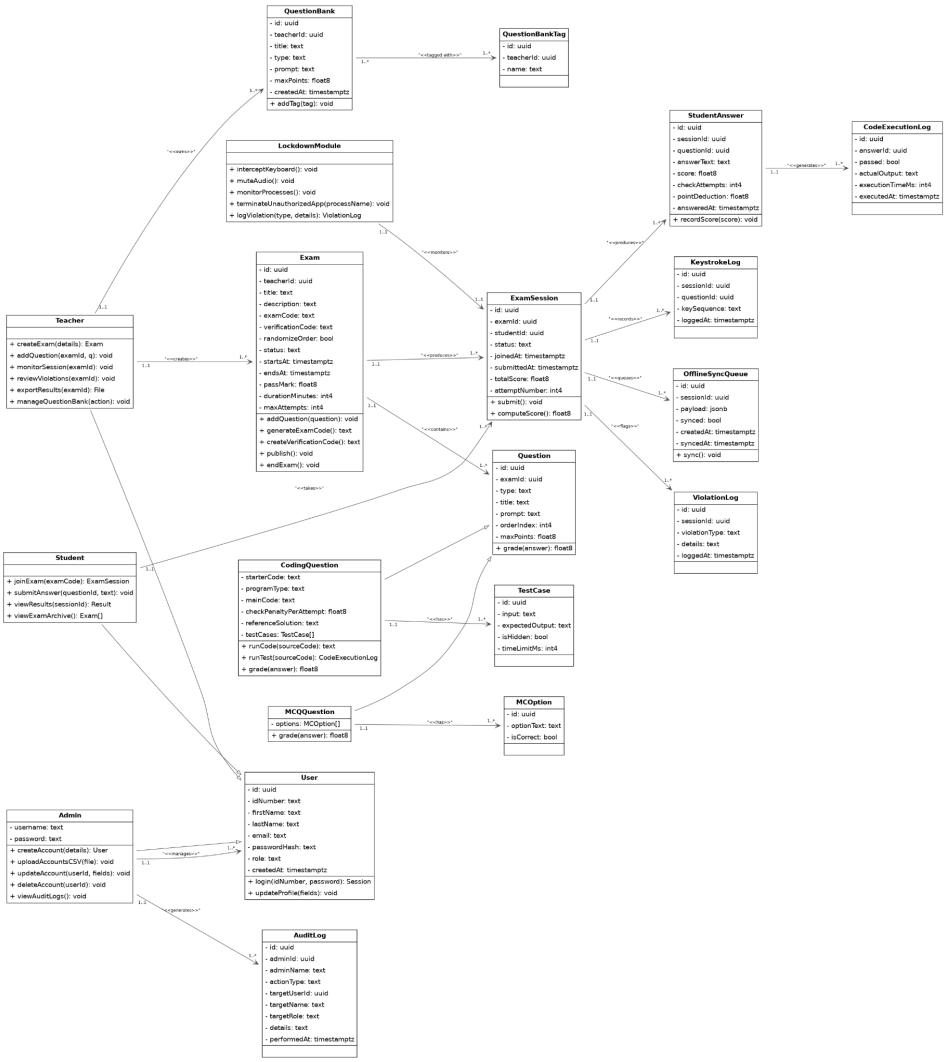


Figure 4.2 - Class Diagram

Figure 4.2 depicts the class diagram of the Contx system, which outlines the key components and their relationships within the platform. The diagram illustrates how the base User class is extended by Student, Teacher, and Admin, each carrying its own set of

responsibilities. The Admin manages user accounts, uploading them individually or through CSV import, and reviews the resulting audit logs. The Teacher creates and manages exams, builds question banks, monitors active sessions, and reviews violations flagged during an exam. The Student joins exams, submits answers, and views results once grading is complete. The Exam class contains Questions, which are specialized into MCQuestion and CodingQuestion, the latter running submitted code independently or validating it against defined test cases. The ExamSession class ties a student to an exam attempt and produces StudentAnswers, KeystrokeLogs, OfflineSyncQueue entries, and ViolationLogs, all of which are monitored by the LockdownModule responsible for intercepting keyboard input, muting audio, and terminating unauthorized applications. These entities are connected through one-to-many and one-to-one relationships to ensure that exam security, code execution, and grading all remain part of a single cohesive data flow.

User Interface Design (Student)

Student Login

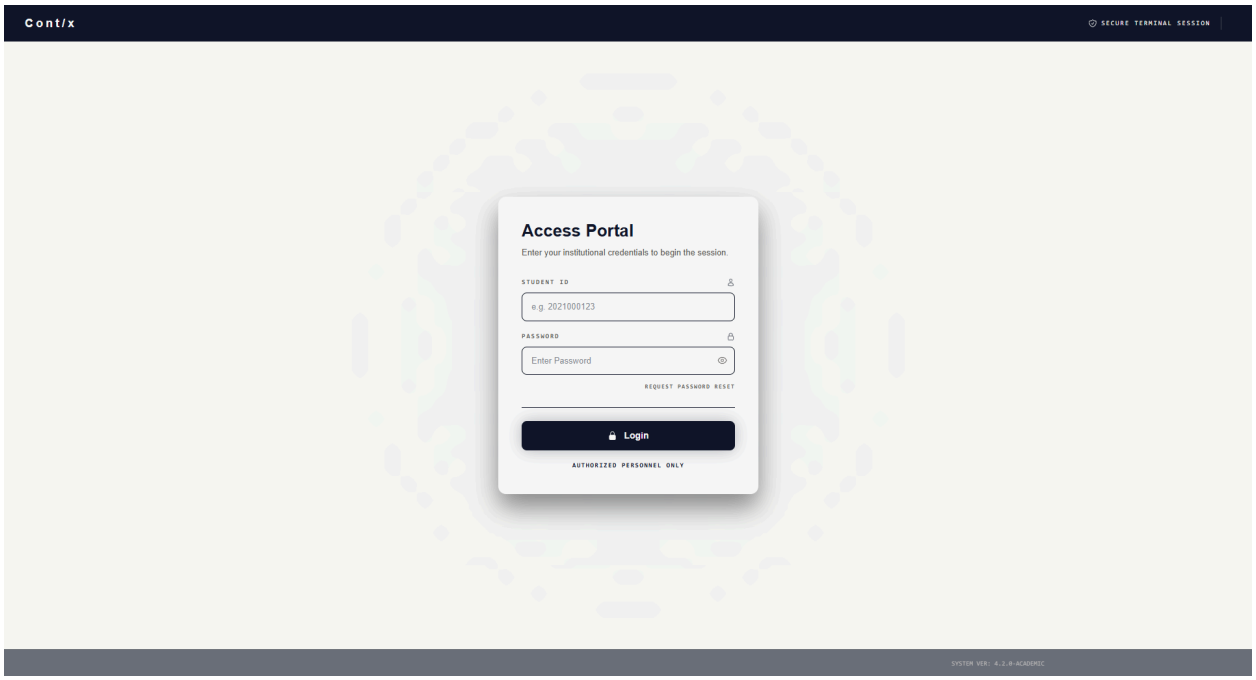


Figure 5.1 - Student Login

Figure 5.1 shows the Student Login screen of the Cont/x desktop client. It displays an "Access Portal" form asking for the student's ID and password, with a password visibility toggle.

The "Secure Terminal Session" label in the header confirms that lockdown is already active at this point, since it engages on launch rather than after login. An "Authorized Personnel Only" note reflects that accounts are created by the admin, not self-registered.

Student Dashboard

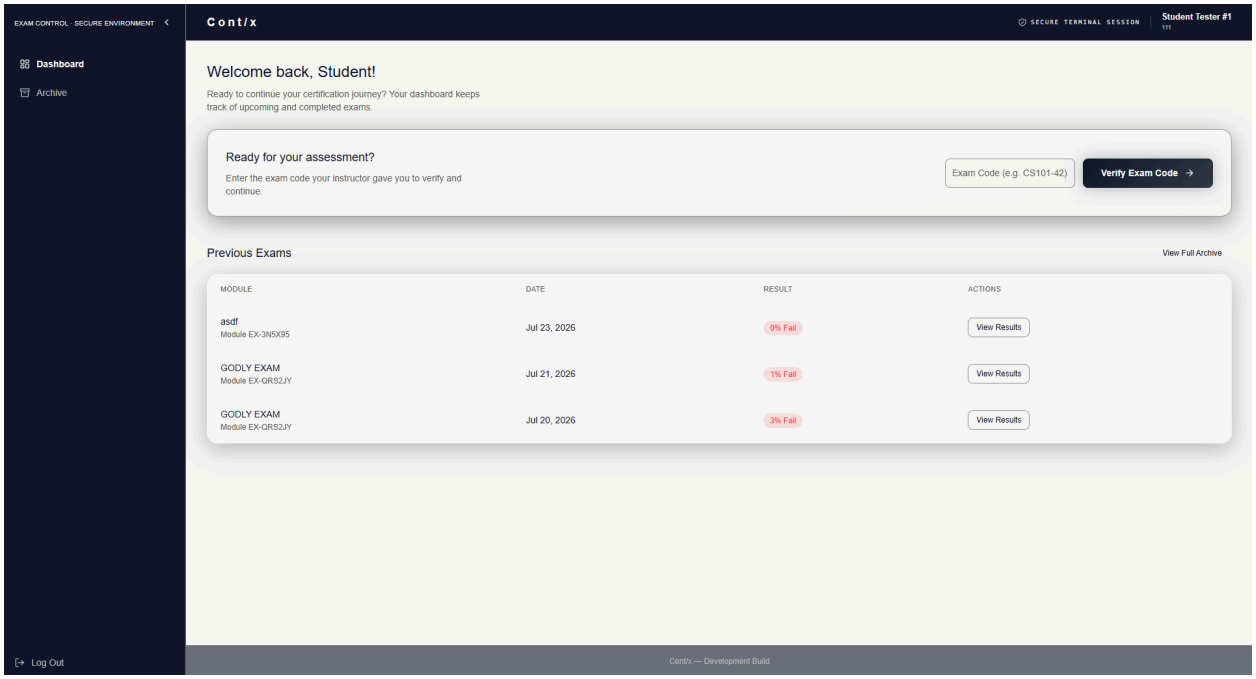


Figure 5.2 - Student Dashboard

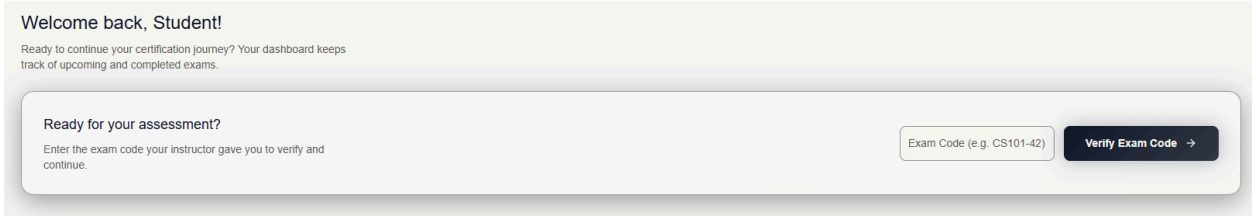


Figure 5.3 - Exam Code

Figure 5.2 and 5.3 shows the Student Dashboard of the Cont/x desktop client. It displays an exam code input where the student enters the code given by their instructor to enter an exam. Below it, a "Previous Exams" table lists the student's 3 latest exams with module name, date taken, result, and a "View Results" action, with a link to view the full archive. The sidebar provides navigation to the Dashboard and Archive sections.

Exam Verification

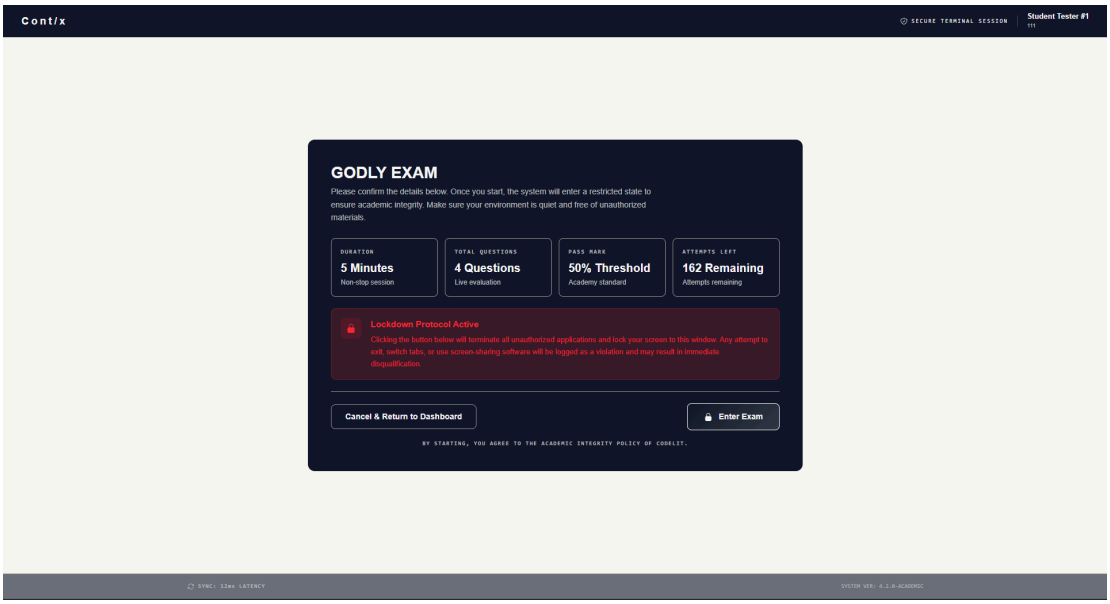


Figure 5.4 - Exam Verification

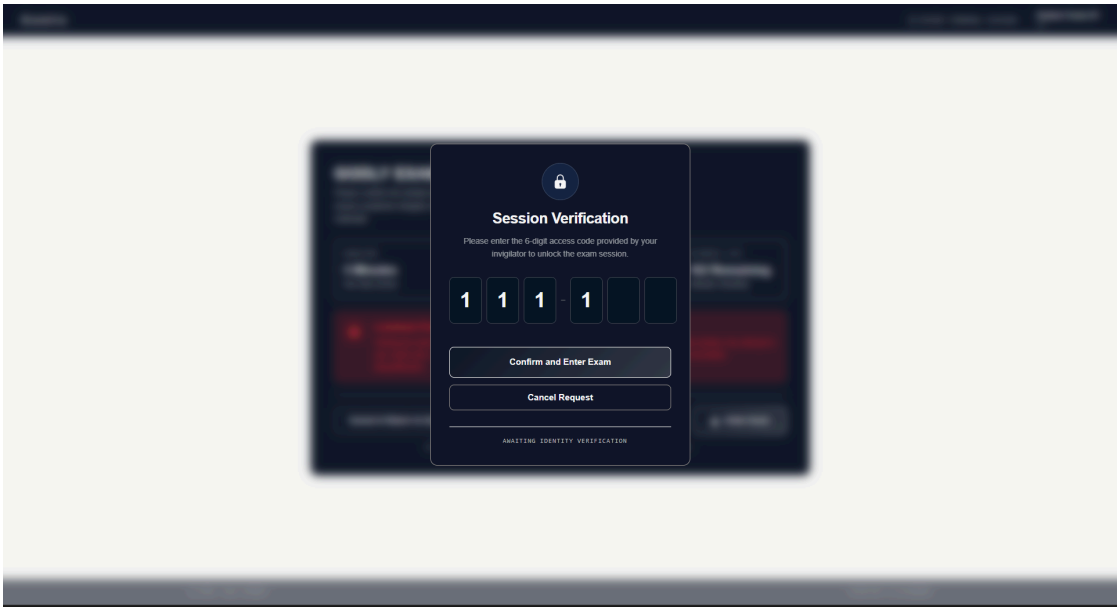


Figure 5.5 - Verification Code

Figure 5.4 and 5.5 shows the Exam Verification screens of the Cont/x desktop client. After the exam code is verified, the system displays the exam summary, duration, total questions, pass mark, and attempts remaining, along with a lockdown warning stating that entering the exam will terminate unauthorized applications and lock the screen to the exam window, with exit

attempts logged as violations. The student may cancel and return to the Dashboard or proceed by clicking "Enter Exam." Proceeding opens a Session Verification modal prompting the student for a 6-digit access code from their instructor. Entering the correct code and clicking "Confirm and Enter Exam" unlocks the session and starts the exam.

Exam Question



Figure 5.6 - Coding Question

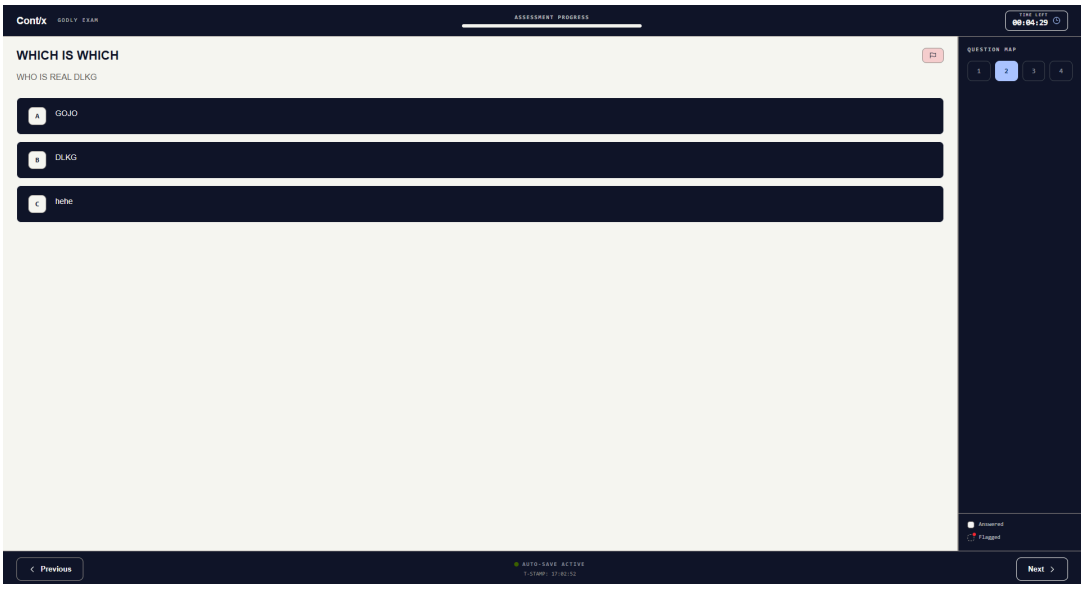


Figure 5.7 - Multiple-Choice Question

Figure 5.6 and 5.7 shows the Exam Screen of the Cont/x desktop client for the question types. All question types share the same layout, a header with the exam title, progress bar, and countdown timer, a Question Map on the right marking questions as Answered or Flagged, and Previous/Next navigation at the bottom with an auto-save indicator.

For coding questions, the center panel shows a Monaco code editor pre-filled with starter code, with "Run Tests" and "Run Code" buttons above it and an output panel below. For MCQ questions, the panel instead displays the question prompt with lettered answer choices the student selects from. Fill-in-the-blank questions follow the same layout as MCQ but present a text input field in place of the answer choices. A flag icon in the top right lets the student mark any question for review before submitting.

Exam Result

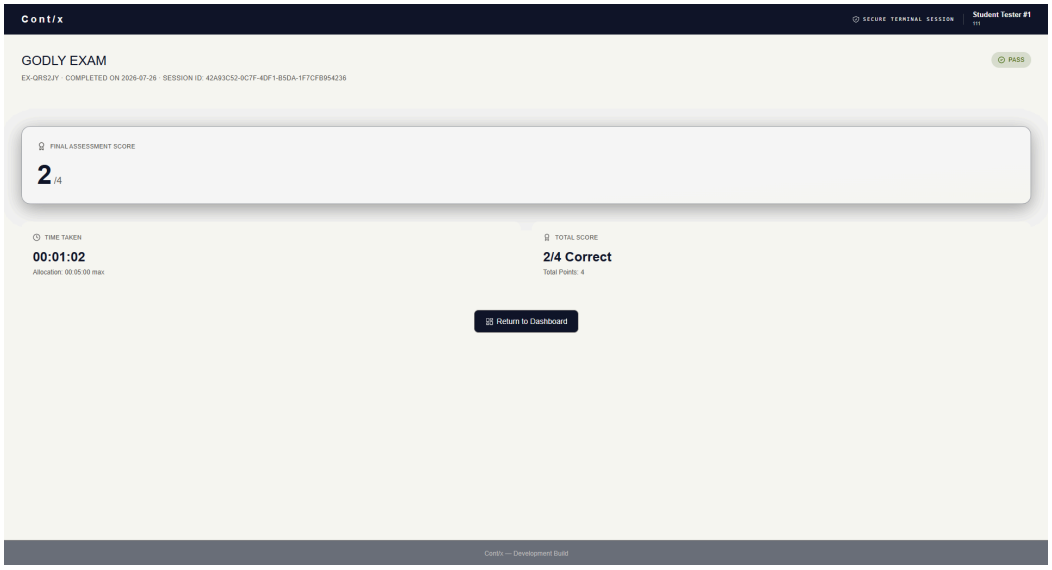


Figure 5.8 - Exam Result

Figure 5.8 shows the Exam Result Screen of the Cont/x desktop client. It displays the exam title, session ID, completion date, and a pass/fail badge. Below it, the final assessment score is shown prominently, followed by the time taken against the allocated duration and the total correct answers out of the exam's total points. A "Return to Dashboard" button lets the student navigate back once they've reviewed their results.

Student Archive

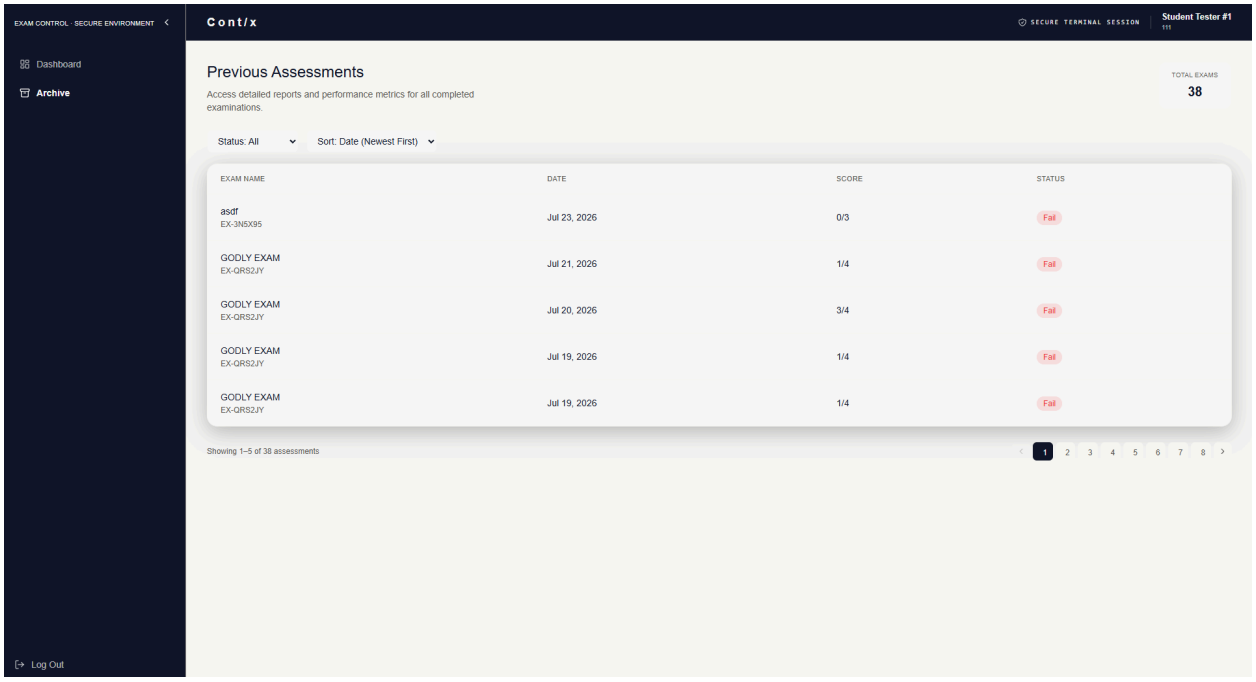


Figure 5.9 - Student Archive

Figure 5.9 shows the Exam Archive screen of the Cont/x desktop client. It lists all of the student's completed exams with exam name, date, score, and pass/fail status, along with a total exam count and status/sort filters. The list is paginated, allowing the student to browse their full exam history beyond the 3 latest exams shown on the Dashboard.

User Interface Design (Teacher)

Teacher Dashboard

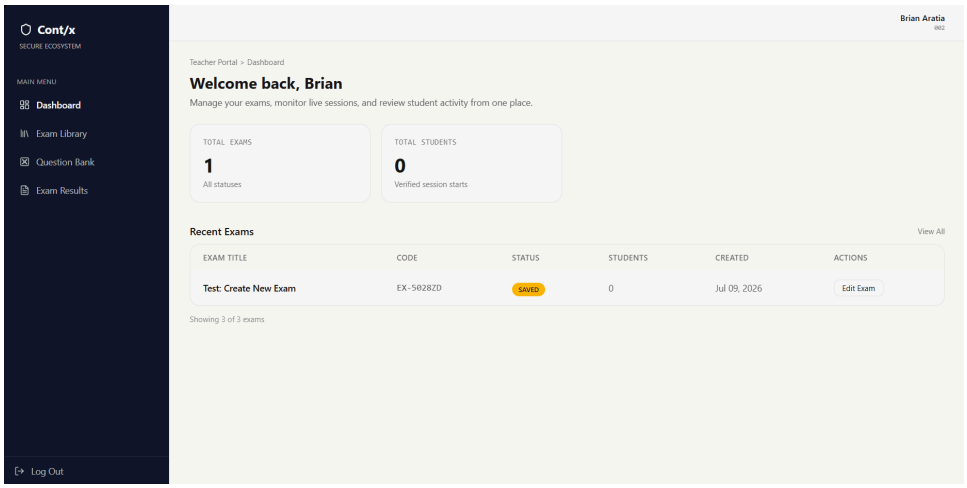


Figure 5.10 - Teacher Dashboard

The Dashboard is the teacher's landing page after logging in. It displays a welcome message with the teacher's name and two summary cards, Total Exams and Total Students, giving an at-a-glance view of overall activity. Below these, a Recent Exams table lists each exam's title, exam code, status (e.g., "Saved"), number of enrolled students, creation date, and quick actions (e.g., "Edit Exam"), with a "View All" link leading to the full Exam Library. This design keeps the teacher's most relevant information visible immediately upon login, without requiring extra navigation.

Exam Library

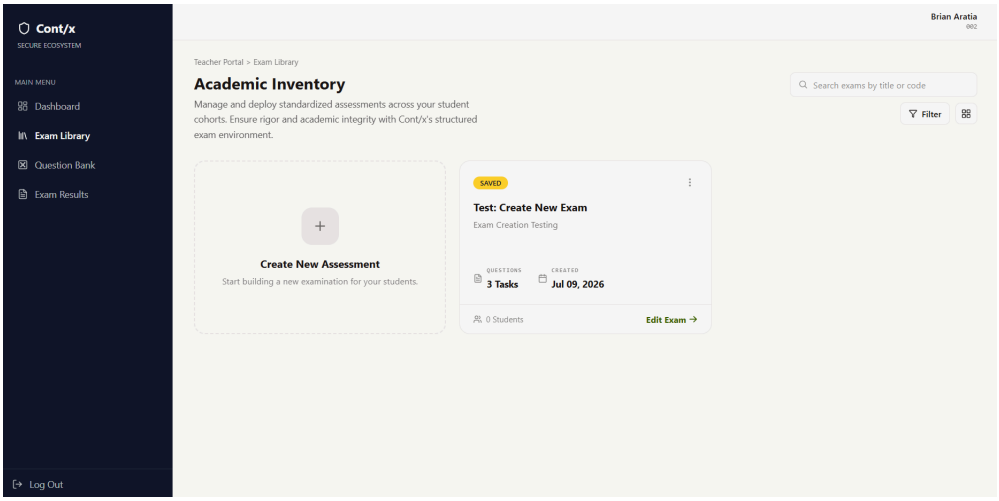


Figure 5.11 - Exam Library

The Exam Library is where the teacher manages all created exams. A search bar allows filtering exams by title or code, and Filter and view-toggle controls sit alongside it. A dedicated "Create New Assessment" card, marked with a plus icon, initiates the exam creation flow. Each existing exam is displayed as its own card showing a status badge (e.g., "Saved"), title, short description, number of questions, creation date, enrolled student count, and an "Edit Exam" link, with a three-dot menu for further actions such as exam settings, publishing or deleting. This card-based layout lets the teacher scan and manage multiple exams at once.

Create New Assessment Modal

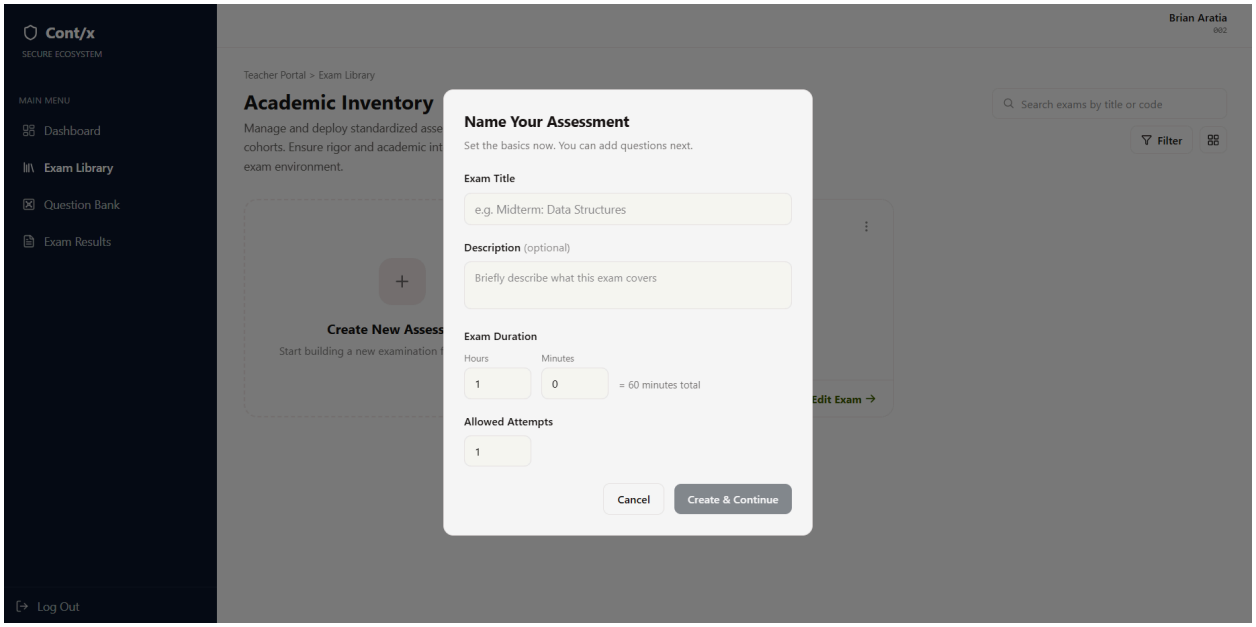


Figure 5.12 - Create New Assessment Modal

This modal appears when the teacher clicks "Create New Assessment" and collects the exam's foundational details before question-building begins. It includes fields for Exam Title, an optional Description, Exam Duration (entered separately as Hours and Minutes, with the total automatically calculated and displayed), and Allowed Attempts. The teacher can cancel or proceed via "Create & Continue," which creates the exam as a draft and moves directly into the Exam Builder. Keeping this step lightweight ensures the teacher can start structuring an exam without being blocked by unnecessary fields.

Exam Builder

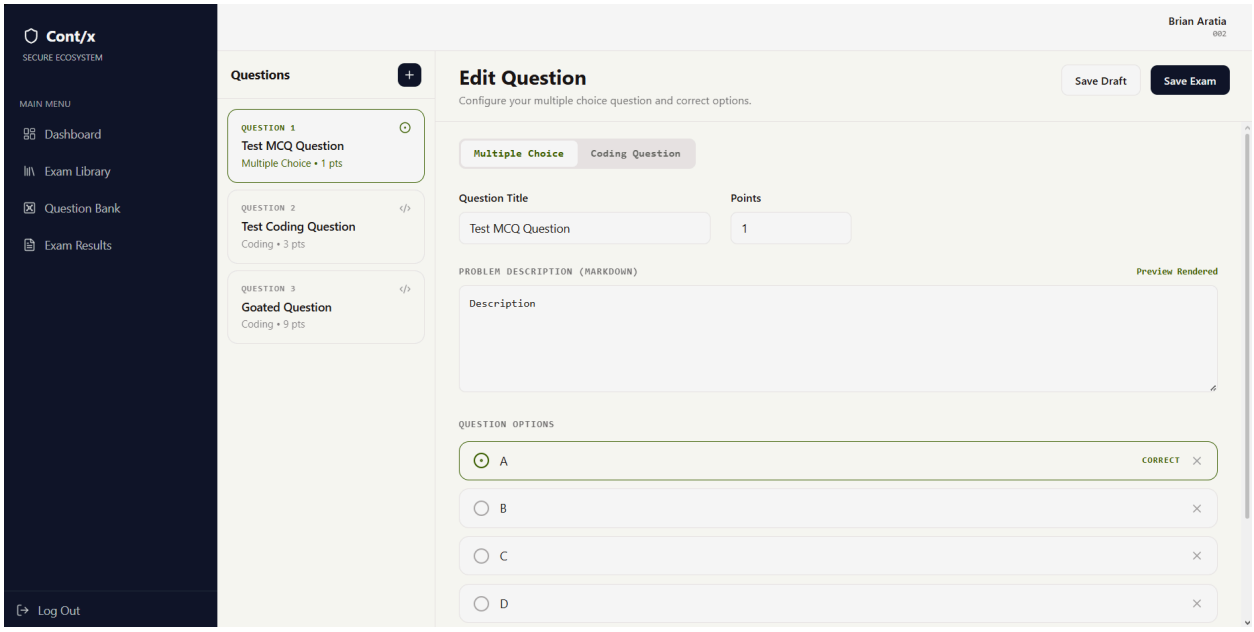


Figure 5.13 - Exam Builder

The Exam Builder is the main workspace for assembling an exam's content. A left-hand panel lists all added questions, each labeled by number, title, type (Multiple Choice or Coding), and point value, with a "+" button to add new ones. The main panel shows the "Edit Question" view for whichever question is selected, including a toggle between Multiple Choice and Coding Question types, a Question Title field, Points field, and a Problem Description field with Markdown support and a Preview/Rendered toggle. For multiple-choice questions, answer options appear with a radio selector to mark the correct answer and an "x" to remove options. "Save Draft" and "Save Exam" buttons in the top-right let the teacher save progress or finalize the question set.

New Question Modal

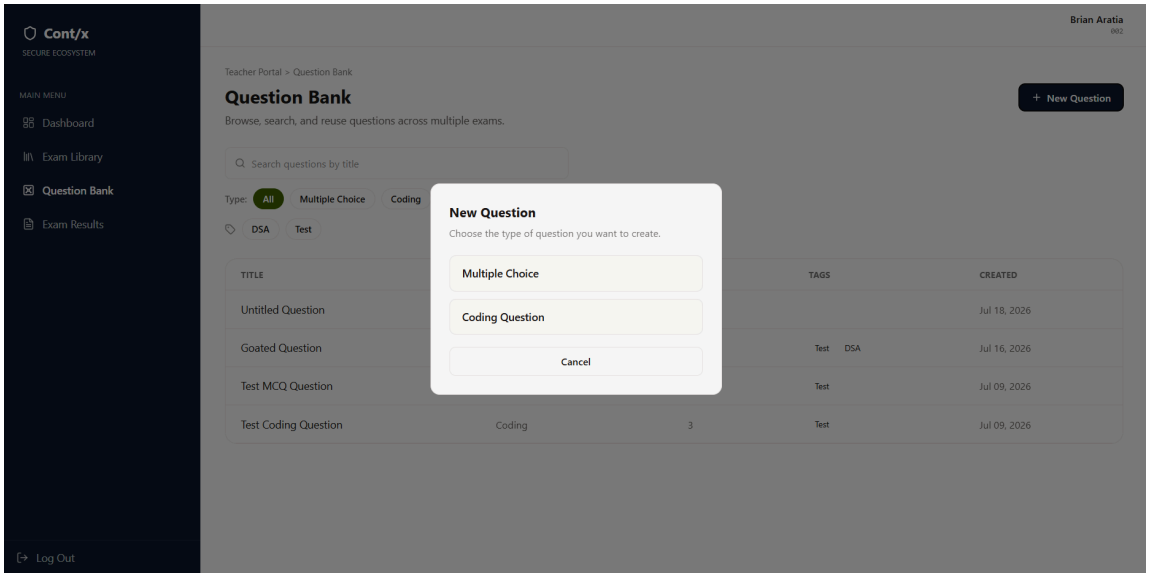


Figure 5.14 - New Question Modal

Triggered by the "New Question" button in the Question Bank, this simple modal asks the teacher to choose between two question types, "Multiple Choice" or "Coding Question", before opening the corresponding editor. A "Cancel" option is also available. This upfront type selection ensures the correct editor and fields are shown immediately, avoiding a generic form that would otherwise need to adapt after the fact.

Question Bank

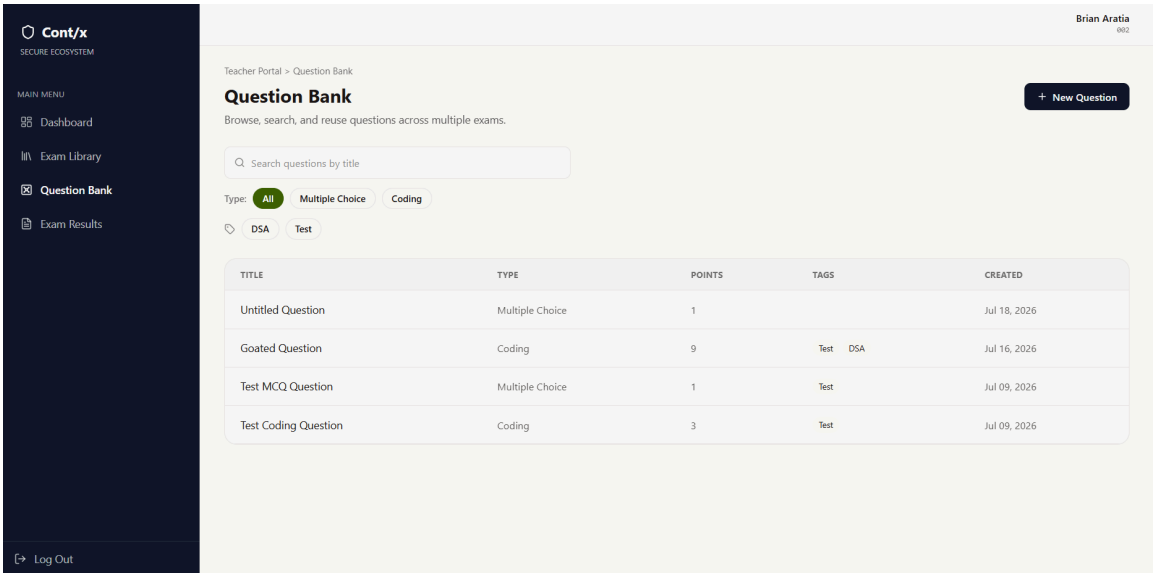


Figure 5.15 - Question Bank

The Question Bank gives the teacher a centralized, reusable library of questions independent of any single exam. It includes a search bar, a Type filter (All, Multiple Choice, Coding), and Tag filters (e.g., "DSA," "Test") for narrowing results. The main table lists each question's title, type, point value, tags, and creation date, with a "New Question" button in the top-right to begin creating a new one. This design supports the teacher's workflow of building a growing bank of questions that can be reused across multiple exams rather than recreated each time.

Create Multiple-Choice Question

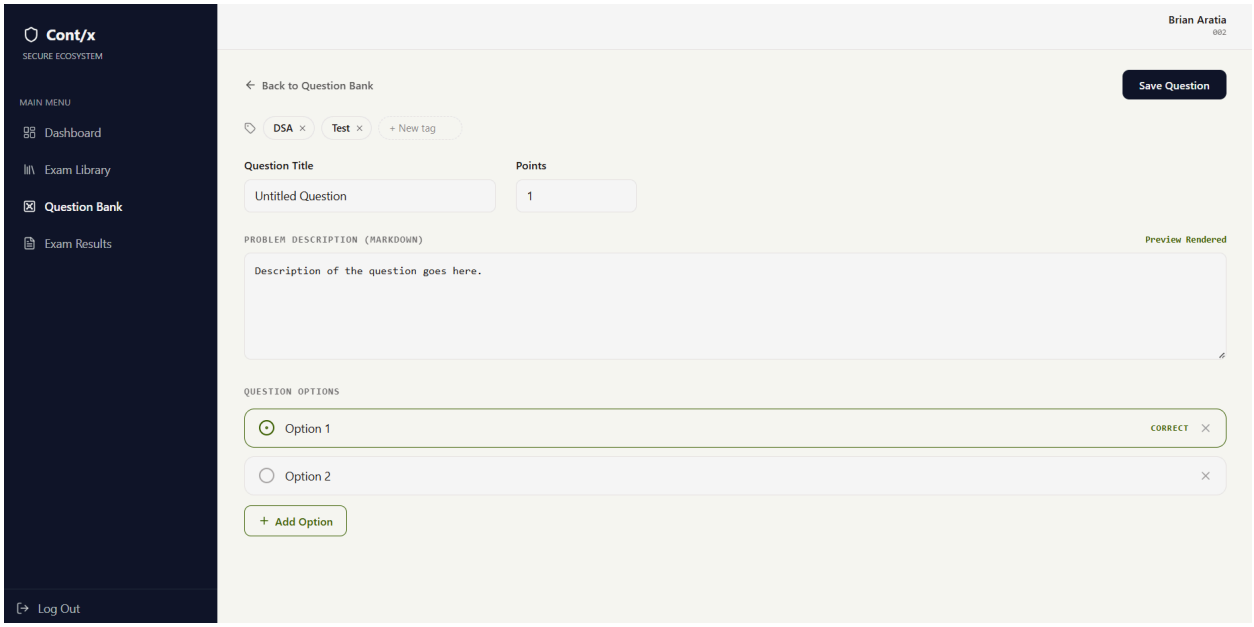


Figure 5.16 - Create Multiple-Choice Question

This screen is used to build or edit a multiple-choice question in the Question Bank. It includes tag chips (with an option to add new tags), a Question Title field, a Points field, and a Markdown-enabled Problem Description with a Preview Rendered toggle. Below this, the Question Options section lists each answer choice with a radio button to mark the correct one, visually flagged with a "Correct" label, an "x" to remove an option, and an "+ Add Option" button to add more. A "Save Question" button in the top-right commits the question to the bank.

Create Coding Question (C Program)

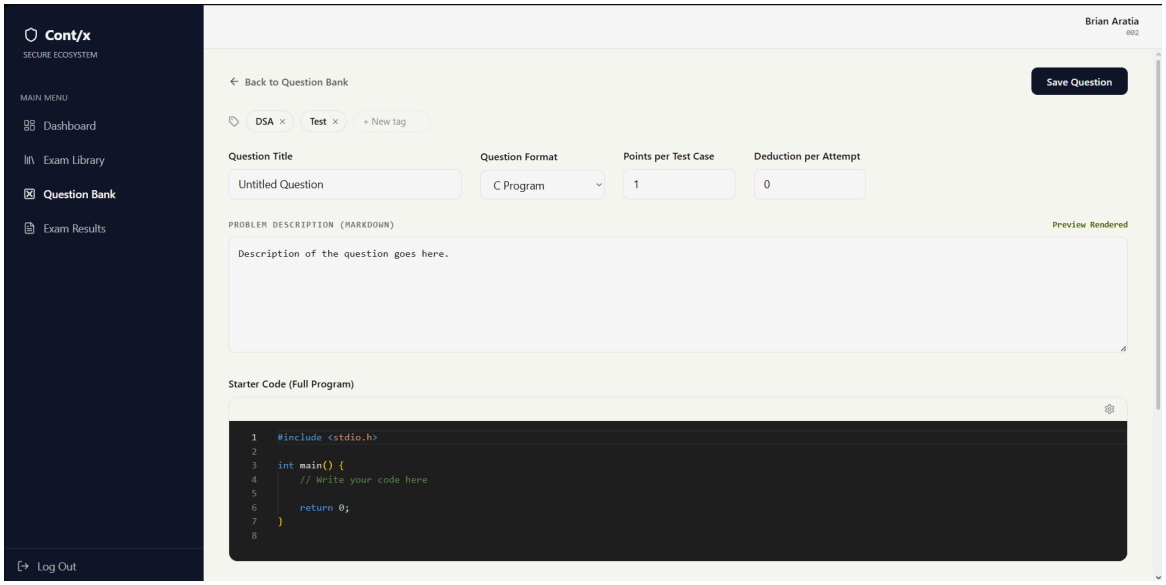


Figure 5.17 - Create Coding Question (C Program)

This is the counterpart editor used when the Question Format is set to "C Program." It shares the same tag, title, points, and description fields as the Function format, but instead of a fixed main-calls-function structure, it provides a Starter Code editor pre-filled with a main() template. This format is suited for questions where the student must write an entire program rather than just one function.

Create Coding Question (C Function)

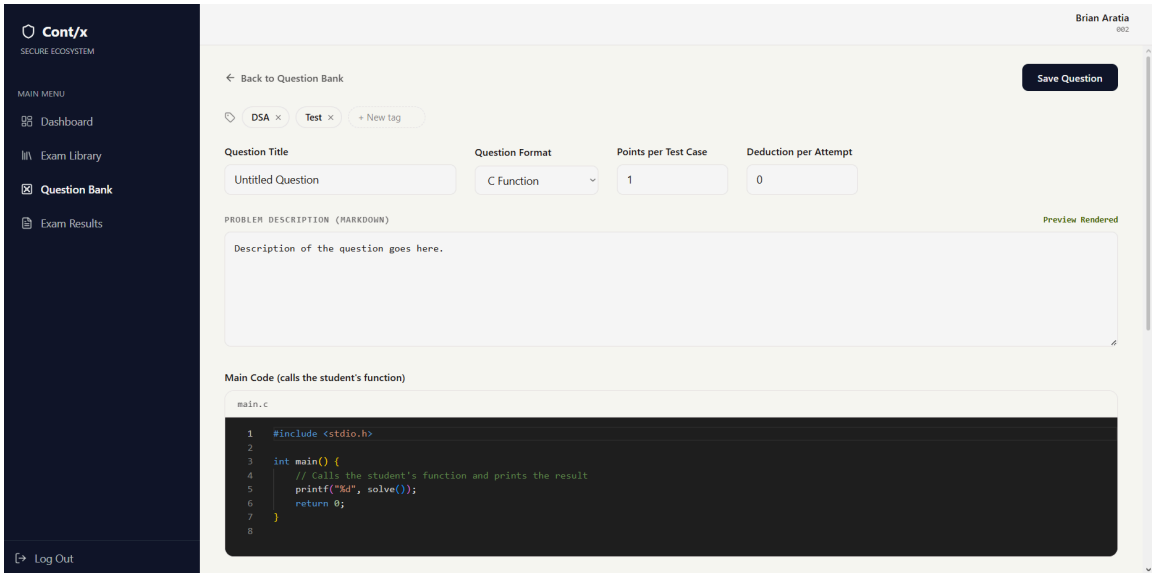


Figure 5.18 - Create Coding Question (C Function)

This variant of the coding question editor is used when the Question Format is set to "C Function." Alongside the shared fields (tags, Question Title, Problem Description), it includes a Question Format dropdown, Points per Test Case, and Deduction per Attempt fields. Because the question expects the student to implement a single function, the editor displays a read-only "Main Code (calls the student's function)" panel, shown here as a main.c file that calls solve() and prints its result, giving the teacher visibility into how the student's function will be invoked and tested.

Exam Published Modal

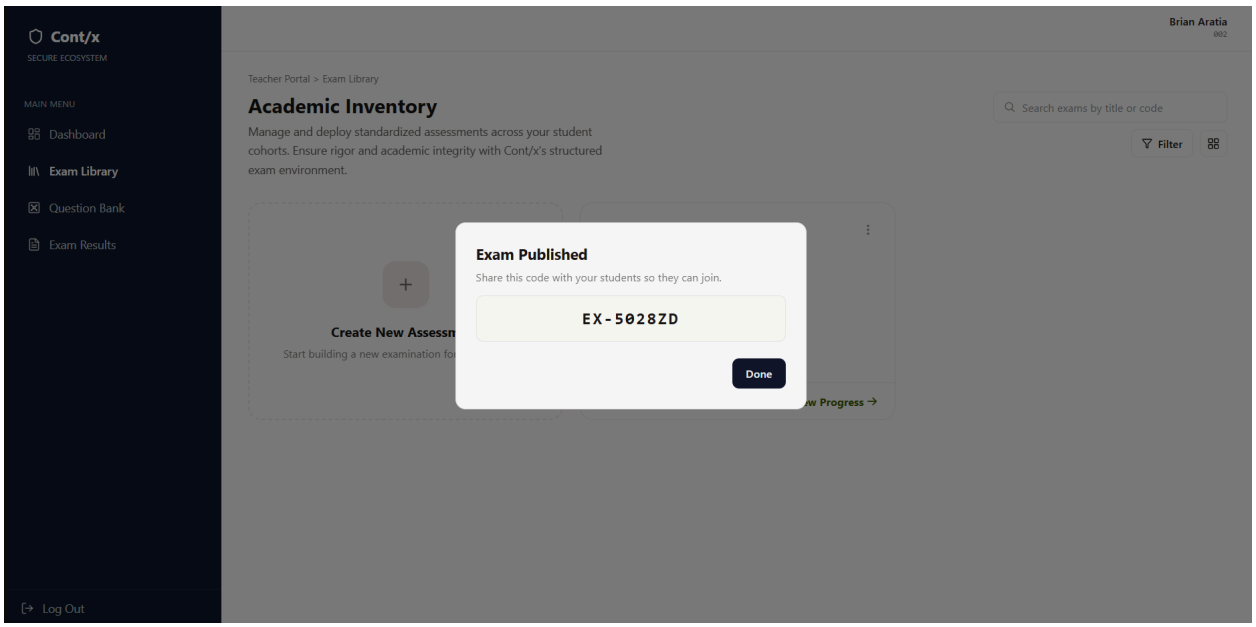


Figure 5.19 - Exam Published Modal

This confirmation modal appears immediately after the teacher successfully publishes an exam. It displays the message "Exam Published" along with the instruction to share the code with students so they can join, followed by the exam's unique code (e.g., "EX-5028ZD") shown prominently in a bordered box for easy copying. A single "Done" button dismisses the modal and returns the teacher to the Exam Library.

Violation Monitoring

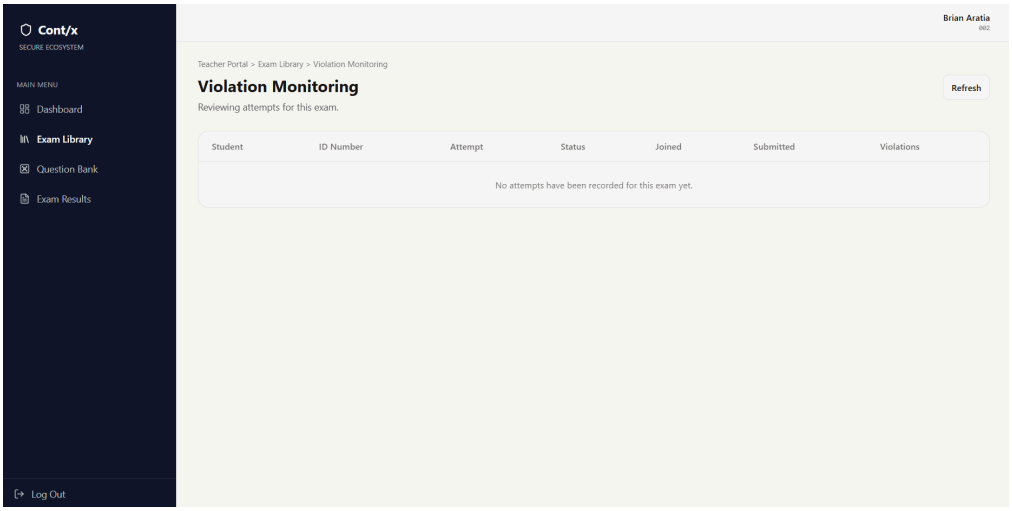


Figure 5.20 - Violation Monitoring

This screen lets the teacher review academic-integrity violations recorded during a specific exam session. It's reached via the Exam Library and displays a table of Student, ID Number, Attempt, Status, Joined time, Submitted time, and Violations columns, giving the teacher a complete picture of each student's session. A "Refresh" button allows the teacher to pull the latest data without reloading the page, which is useful while an exam is still active. When no student has yet attempted the exam, the table displays "No attempts have been recorded for this exam yet," as shown here.

Exam Results

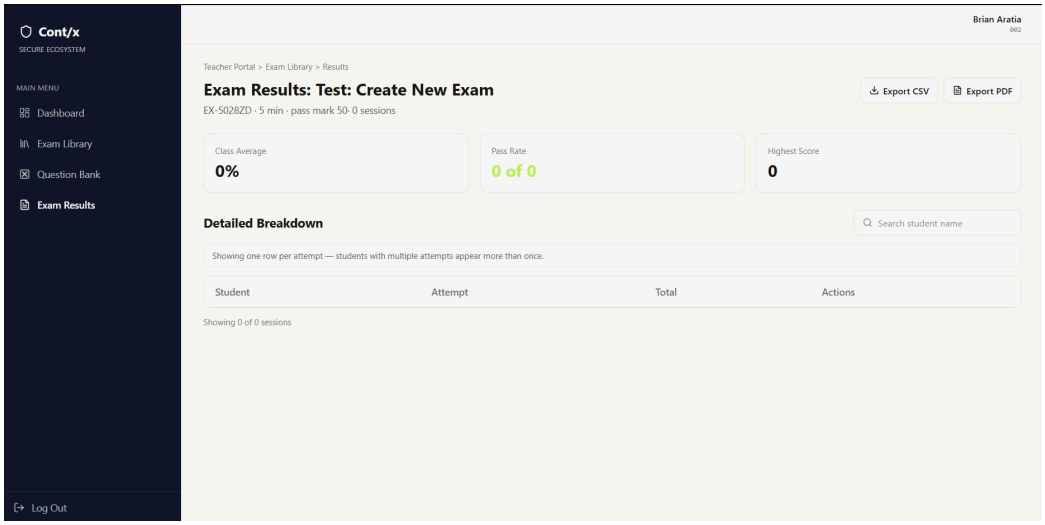


Figure 5.21 - Exam Results

This screen displays the outcome of a completed exam, headed by the exam's title along with its code, duration, pass mark, and total session count. Three summary cards show Class Average, Pass Rate, and Highest Score at a glance. Below this, a "Detailed Breakdown" section includes a search bar for finding a specific student and a table listing Student, Attempt, Total, and Actions. "Export CSV" and "Export PDF" buttons in the top-right let the teacher download the results in either format.

User Interface Design (Admin)

Admin Login

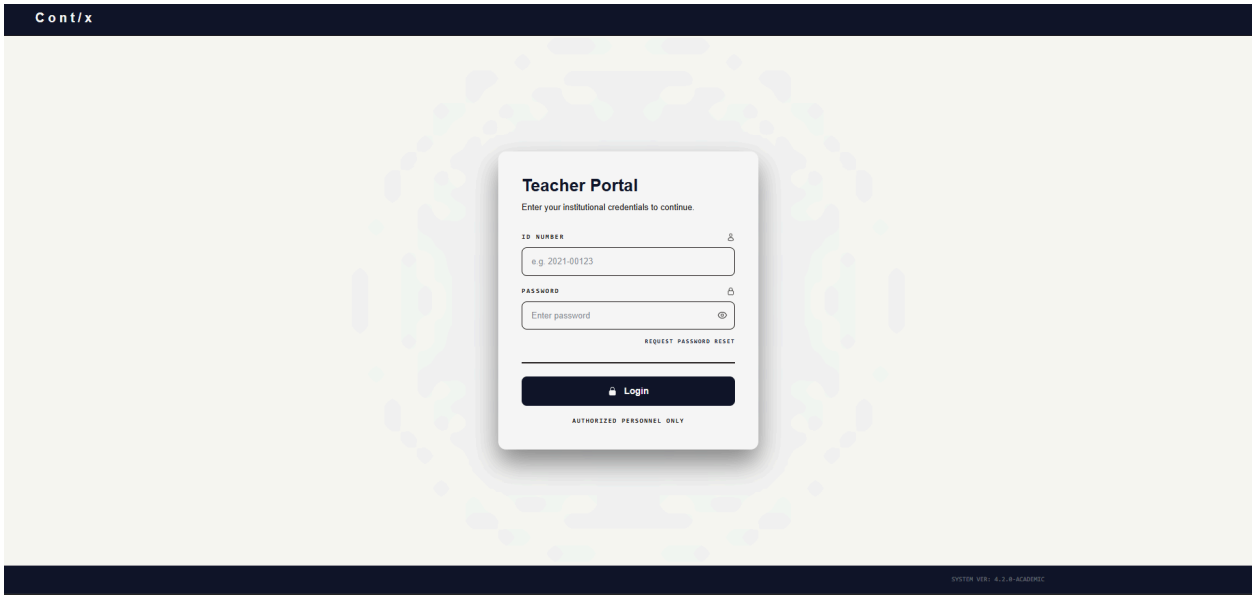


Figure 5.22 Admin Login

Figure 5.22 shows the Login screen of the Cont/x web portal, shared by both Admin and Teacher. It displays a form requesting an ID Number and password, with a password visibility toggle and a "Request Password Reset" link. An "Authorized Personnel Only" label reflects that accounts are created by the admin rather than through self-registration. Since this is the same shared component used by Teacher, the card title reflects whichever role is logging in, "Admin Portal" for the admin, "Teacher Portal" as shown here for the teacher, while both route to their respective dashboards after authentication.

Dashboard - User Management

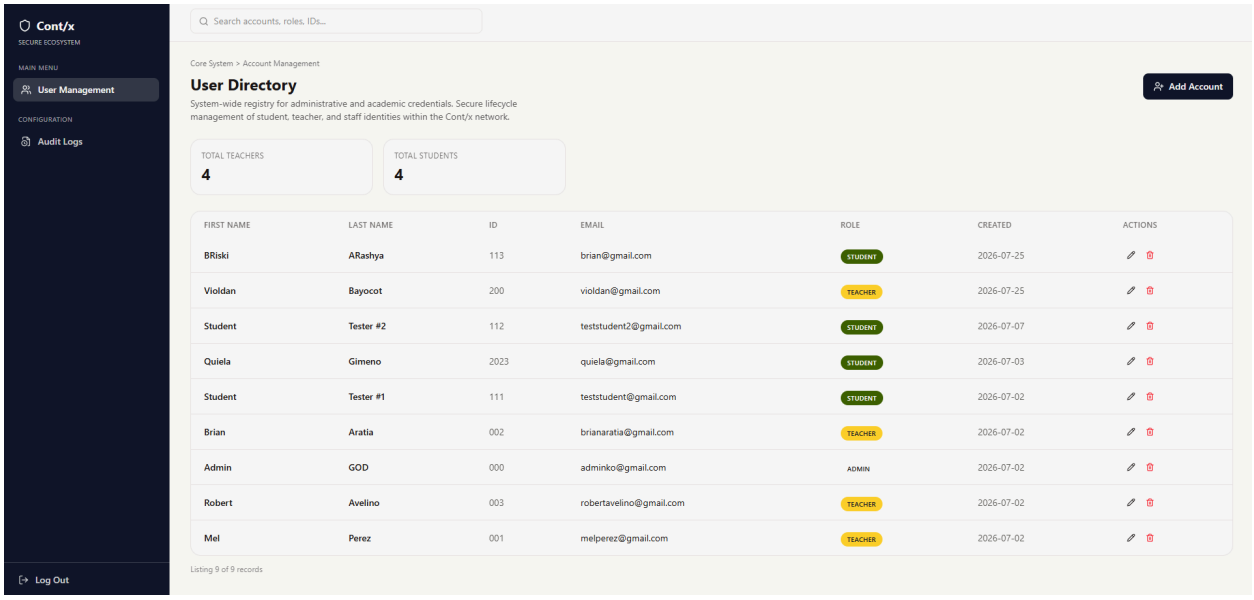


Figure 5.23 Dashboard - User Management

Figure 5.23 shows the User Management screen of the Cont/x web portal. It displays a directory table listing all teacher and student accounts with their name, ID, email, role, and date created, along with a search bar and total teacher/student counts. An "Add Account" button opens the account creation flow, while each row has Edit and Delete actions for updating or removing an account. The sidebar also links to Audit Logs, keeping account management and activity tracking under the same navigation.

Create Account

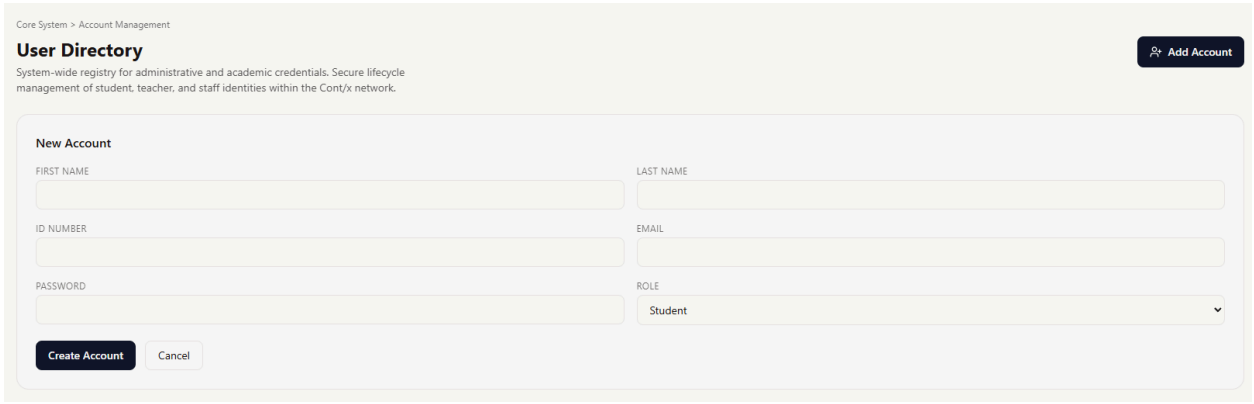


Figure 5.24 Create Account

Figure 5.24 shows the Create Account screen of the Cont/x web portal, accessed via the "Add Account" button on User Management. It presents a "New Account" form requesting First Name, Last Name, ID Number, Email, Password, and Role, with the role defaulting to Student. The admin submits with "Create Account" or exits with "Cancel." This screen currently supports manual account creation only; a CSV upload option for bulk account creation, as described in the Create Account via CSV Upload use case, is planned but not yet reflected in this mockup.

Admin Audit Log

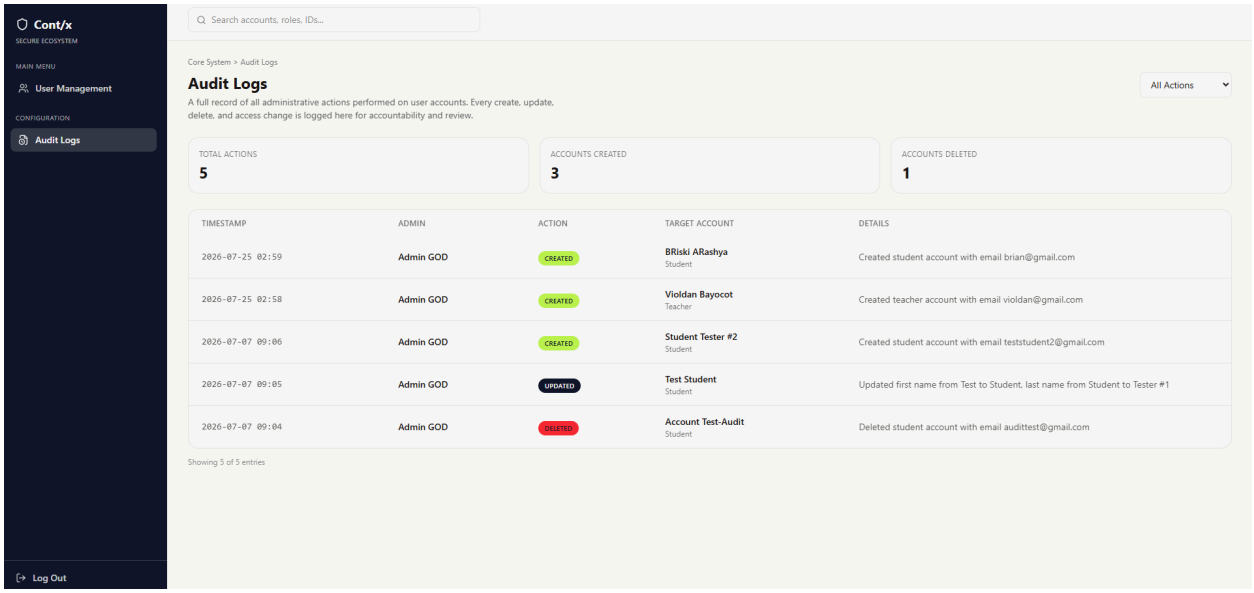


Figure 5.25 Admin Audit Log

Figure 5.25 shows the Audit Logs screen of the Cont/x web portal, accessible only to Admin accounts. It displays a full record of administrative actions performed on user accounts, each entry showing the timestamp, admin who performed it, action type (Created, Updated, or Deleted), target account, and a description of the change. Summary counters at the top show total actions, accounts created, and accounts deleted, with a filter to narrow the log by action type. This screen supports the View Audit Logs use case, keeping account management traceable and restricted to Admin visibility only.

References

[1] Z. Staff, "Ten strategies for preventing cheating in coding class," zyBooks Guide, Jul. 16, 2025. [Online]. Available: https://www.zybooks.com/ten-strategies-to-preventing-cheating-in-coding-class-zybooks guide/

[2] D. R. E. Cotton, P. A. Cotton, and J. R. Shipway, "Chatting and cheating: Ensuring academic integrity in the era of ChatGPT," Innovations in Education and Teaching International, vol. 61, no. 2, pp. 228–239, 2023, doi: 10.1080/14703297.2023.2190148.

[3] S. M. S. Karim, S. Parvin, C. B. U. Ahmed, and M. T. Hossain, "Teachers' and students' perceptions of online education in Bangladesh: Challenges and solutions," Discover Education, vol. 4, no. 1, 2025, doi: 10.1007/s44217-025-00653-w.

[4] S. Combéfis, "Automated code assessment for education: Review, classification and perspectives on techniques and tools," Software, vol. 1, no. 1, pp. 3–30, 2022, doi: 10.3390/software1010002.

[5] Respondus, "LockDown Browser - Respondus," May 27, 2026. [Online]. Available: https://web.respondus.com/he/lockdownbrowser/

[6] Safe Exam Browser, "Safe Exam Browser - About," SEB. [Online]. Available: https://safeexambrowser.org/about_overview_en.html

[7] Proctorio, "Securing the integrity of online assessments," Proctorio Inc. [Online]. Available: https://proctorio.com/

[8] Proctorio, "Products," Proctorio Inc. [Online]. Available: https://proctorio.com/products

[9] Proctorio, "ProctorioPro," Proctorio Inc. [Online]. Available: https://proctorio.com/pro

[10] M. Gaffney, "Secure online proctoring for education: The threat of remote access," PSI Exams, Aug. 16, 2024. [Online]. Available: https://www.psiexams.com/knowledge-hub/secure-online-proctoring-for-education-the-threat-of-remote-access/

[11] J. J. A. Maranga, L. K. J. Matugas, J. F. W. Lim, and C. L. C. S. Romana, "CodeChum: An online learning and monitoring platform for C programming." [Online]. Available: https://eric.ed.gov/?id=ED601226

[12] R. Lobb and J. Harlow, "CodeRunner," ACM Inroads, vol. 7, no. 1, pp. 47–51, 2016, doi: 10.1145/2810041.

[13] A. Pal, "Lockdown browser: Best features, benefits and limitations," Diversified Business Solutions Pvt. Ltd., Apr. 21, 2026. [Online]. Available: https://examonline.in/what-is-lockdown-browser/